V ⊗ W
V ⊕ W
A ↓ B
∏
∑_n
⌊x⌋
⌈x⌉

Components

X	object
f: X → Y	arrow
A	category
F: A → B ∈ B^A	functor i.e. a path
B^A = { A → B }	functor category
A(X, Y) ∈ Set	hom set
A(-, Y): A → Set ∈ Set^A	hom functor
φ: F ⇒ G	natural transformation
Nat(F, G) = { F ⇒ G }	

==========

----- Category -----

Category vs functor vs natural transformation

From "Category theory for the working mathematician" [Section 1.4 Natural transformation]:

'As Eilenberg-Mac Lane first observed,
"category" has been defined in order to be able to define "functor",
and "functor" has been defined in order to be able to define "natural transformation"'

A category is just a **space** of things.
Functors pick out **parameters** of the space to compare, and natural transforms provide a notion of **comparison** relative to the parameters.
i.e.
A **category** is a space of binary-like **relationships (morphisms**)

f ∘ g

A **functor** is a **parameterization** (~ subcategory) of a category **(homomorphism**)

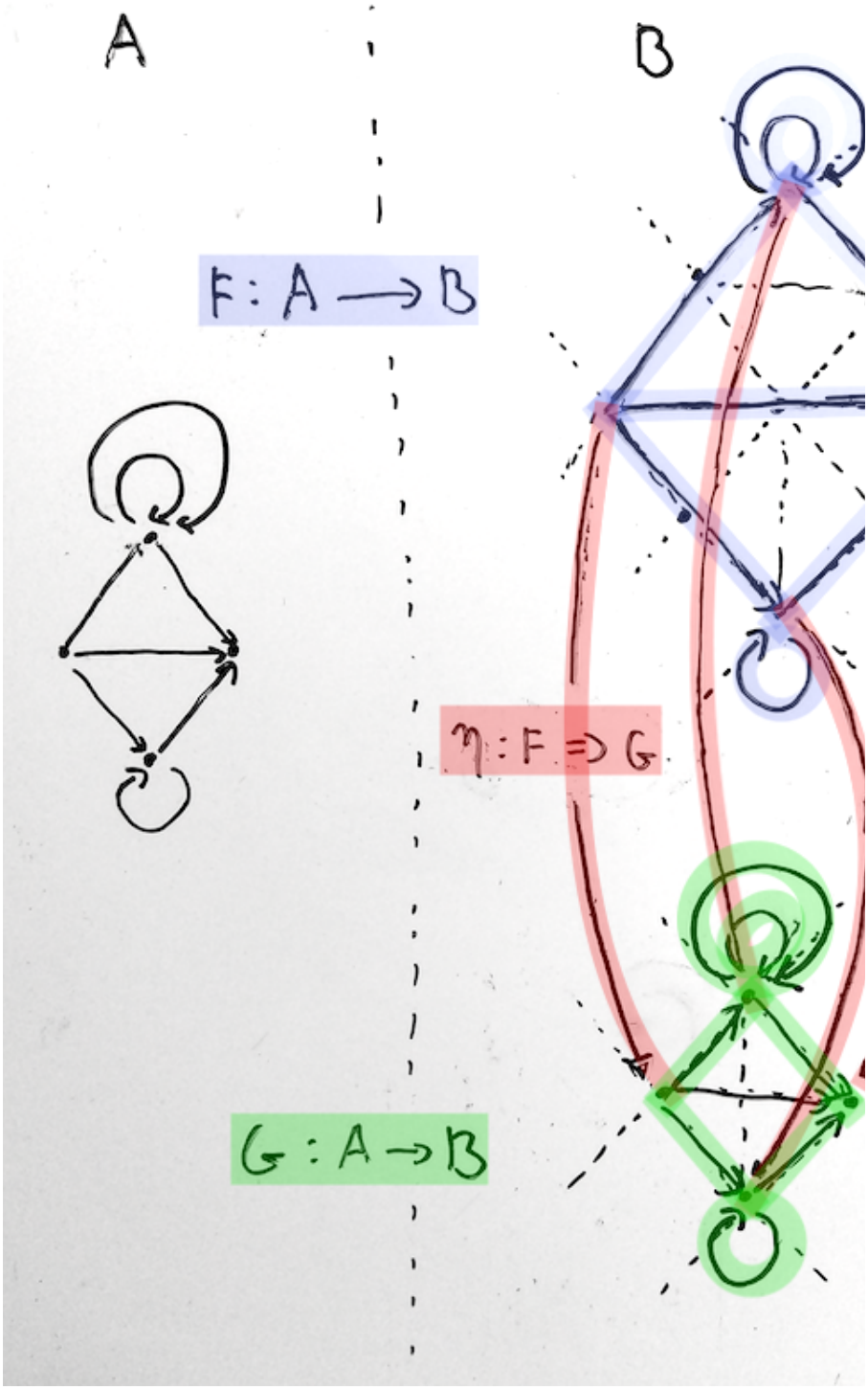
F: A → B B parameterized by A

FA ⊆ B **Subcategory**

A natural transform provides a way to **compare** (a form of **equivalence**) the parameterizations

η: F ⇒ G

η_a: Fa → Ga Fa is equivalent to Ga



*** Category**

Category theory

Category theory is a study of **morphisms**.

Morphism

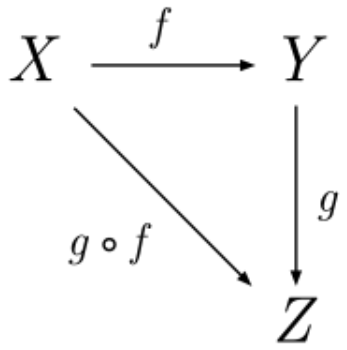
Let X, Y be **objects**.
Then

f: X → Y

is a **morphism f between X and Y**

hom(X, Y) is the set of all morphisms between X and Y.
i.e.

hom(X, Y) = { f | f: X → Y }



Category
A category is essentially a set + a **partial binary operation** (i.e. not defined for all pairs) + left and right **identity**

g ∘ f if it exists
f ∘ 1X = f
1Y ∘ f = f

Definition:
A **category** C consists of 2 things:

- a class **obj(C)** of **objects**

- a class **hom(C)** of **morphisms** (aka arrows, maps)

hom(C) = { f | f ∈ hom(X,Y) X, Y ∈ obj(C) }
{ f | f ∈ X → Y X, Y ∈ obj(C) }

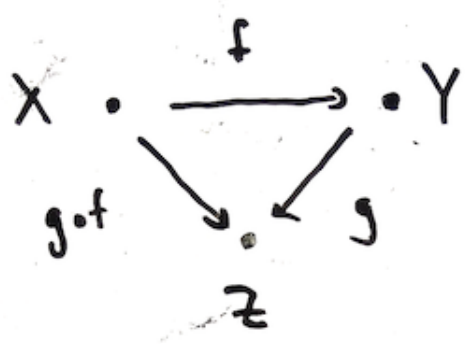
- a **binary operation** (composition) of morphisms

g ∘ f : X → Z where f: X → Y
g: Y → Z

satisfying the rules

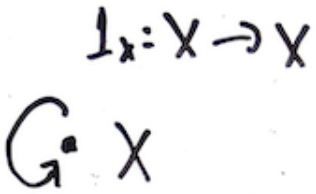
rule 1) **associativity**:

h ∘ (g ∘ f) = (h ∘ g) ∘ f where f: W → X
g: X → Y
h: Y → Z



rule 2) **identity**: for every object X in C, there exists id_X such that for any morphism f,g

left identity:	**1X ∘ f = f**	where	f: Y → X
right identity:	**g ∘ 1X = g**		g: X → Z



e.g.
ℝ is a category

obj(ℝ) = any subset of ℝ
e.g.
{ 1 } ∈ obj(ℝ)
{ 1, 2 } ∈ obj(ℝ)
[0, 1] ∈ obj(ℝ)

hom(ℝ) = function

e.g.
ℤ is a category

obj(ℤ) = any subset of ℤ
hom(ℤ) = function

*** Types of morphisms**

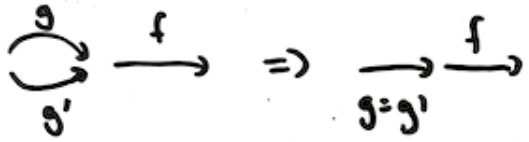
idempotent f: X → X f ∘ f = f

monomorphic arrow f: X → Y **f ∘ g** is **unique** for all g
Generalization of **injectivity**

i.e. g, f ∘ g = f ∘ g' => g = g'

monic f ∘ g is unique for all g

injective f(x) is unique for all x



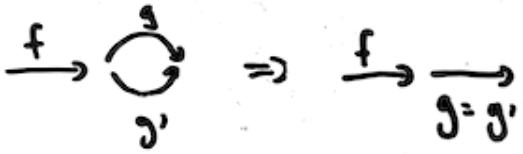
epimorphic arrow f: X → Y g ∘ f is **unique** for all g

Generalization of **surjectivity**

i.e. g ∘ f = g' ∘ f => g = g'

epimorphic g ∘ f = g' ∘ f => g = g'

surjective g(f(x)) = g'(f(x)) for all x => g = g'



isomorphism f: X → Y

isomorphic objects X ≅ Y there exists f: X → Y, f^-1: Y → X with

f f^-1 = 1X f^-1 f = 1Y

Note: isomorphic => **monomorphic** + **epimorphic**

fg = fg' => g = f^-1 fg = f^-1 fg' = g' i.e. monic

gf = g'f => g = gf f^-1 = g'f f^-1 = g' i.e. epi

split monic h: Y → X A pair of functions **g ∘ h = 1**: Y → Y

Generalization of **injective**+**surjective pair** of an identity

aka **left invertible** **f = h ∘ g** is called a **split**

g ∘ h = 1: Y → Y <=> h injective

g surjective

split epic g: X → Y

aka **right invertible**

Properties:

a) f **idempotent**: f f = hghg = hg = f

b) g is **left inverse** of h

h is **right inverse** of g

b) h is **monic**, g is **epic**

hk = hk' => k = ghk = ghk' = k' i.e. monic

kg = k'g => k = kgh = kgh' = k' i.e. epi

initial object X unique arrow X → Y for every Y

Generalization of **min/max**

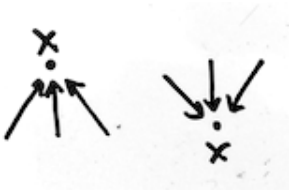
no arrow Y → X

e.g. ∅ in Set is a null

terminal object X unique arrow Y → X for every Y

1 (group with only identity) in Grp is a null

no arrow X → Y



zero (null) object 0 both initial and terminal

Generalization of a zero

zero arrow: X → 0 → Y unique arrow passing through 0

e.g.

X → 0 → Y

∅ Set

{1} Grp



Proposition: left/right inverse

f has **left inverse + epic** <=> **isomorphic**

i.e. g ∘ f = 1

f has **right inverse** + **monic** <=> **isomorphic**

i.e. f ∘ g = 1

f has **left + right inverse** <=> **isomorphic**

i.e. g ∘ f = 1 **g = g'** **unique inverse**

f ∘ g' = 1 i.e. left + right inverses are equal

Corollary:

f, g are **monic** and f ∘ g ∘ f = f <=> isomorphic

f, g are **epic** and f ∘ g ∘ f = f <=> isomorphic

==========

Proof:

- left inverse + epic:

Let f: X → Y be left invertible

i.e.

g ∘ f = 1

=> f = f ∘ (g ∘ f)

= (f ∘ g) ∘ f

and

f = 1 ∘ f

=> f ∘ g = 1 since f is epic

- right inverse + monic: similar

- left + right inverse:
Just compose **both sides**

g = g ∘ (f ∘ g')
= (g ∘ f) ∘ g'
= g'
=> -g = g'

- corollary:

f ∘ g ∘ f = f = f ∘ 1
=> g ∘ f = 1 since f is monic

QED
==========

Proposition: Set
For a function f: X → Y in Set

f is **injective**	<=>	f is **monomorphic**
f is **surjective**	<=>	f is **epimorphic**

==========
Proof:
a) injection
Suppose f is injective. Then

f(g(x)) = f(g'(x))
=> g(x) = g'(x) injectivity
=> g = g' since for any x

Conversely, if f is not injective, choose x ≠ x' but f(x) = f(x')
Let g(1) = x, g'(1) = x'

b) surjection
Suppose f is surjective. Then for any y ∈ dom(g)

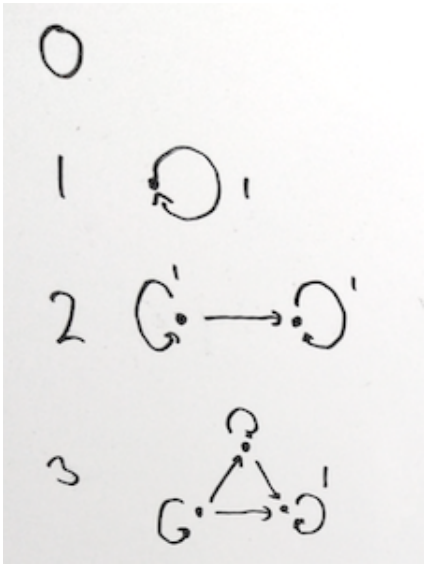
y = f(x) for some x
=> g(y) = g(f(x)) = g'(f(x)) = g'(y) since gf = g'f
=> g = g'

Conversely, suppose f isn't surjective, so there exists y ∈ dom(g)/Im f.
Let g(y) = 1 g'(y) = -1

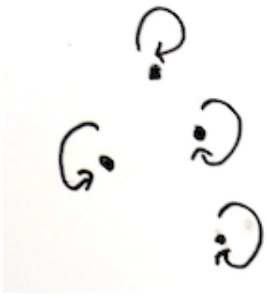
QED
==========

<u>* Common categories</u>
Common categories

	morphism composition	**object**
0	None	
	-	None
1	I	
	-	1 object X
	1 object with identity	
2	X → Y, 1a, 1b	
	-	2 object X, Y
	2 objects with 1 morphism in between	
3	X → Y, Y → Z, X → Z	
	Y → Z X → Y = X → Z	3 objects X Y Z
	3 objects + 1 morphism for each pair	



Discrete	1X for each X	
	-	any object X
	No morphism between objects	



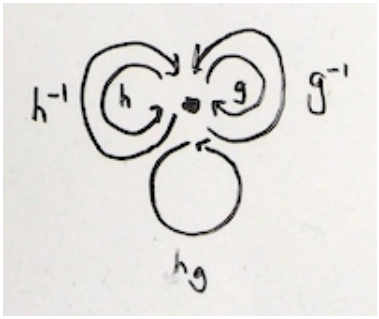
Monoid	X → X
	1 object + morphisms on self



Group G	g: X → X, g^-1: X→ X

group operation g . h G = { X } X is any dummy

A single group Each morphism g is a **group element**

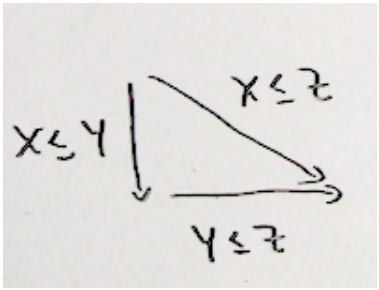


Preorder X <= Y

X <= Y <= Z an object X

Partial order Partial order if X <= Y and Y <= X imply X = Y

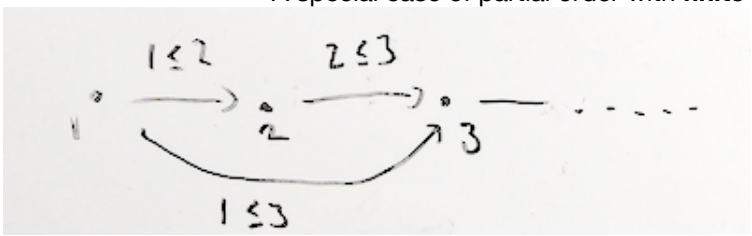
Linear order Linear order if X <= Y or Y <= X for any X, Y



Ordinal n <= m

n <= m <= o a number n

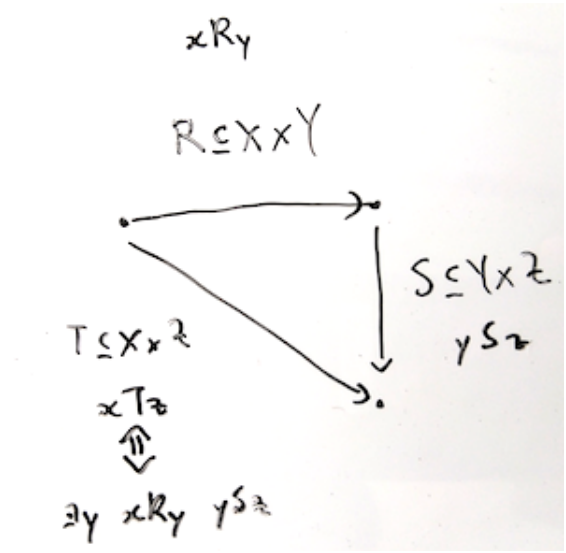
A special case of partial order with **finite numbers**



Rel R ⊂ X x Y

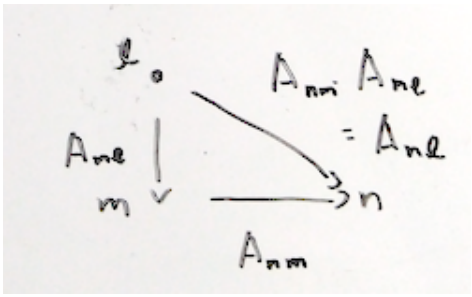
relation composition S ∘ R a set X

Binary relation xRy



Matr_K A_nm: : m → n
matrix multiplication a positive integer m
Each morphism is a **n x m matrix in a field K**
i.e. A_nm ∘ A_ml = A_nm A_ml

= n x l matrix



Set f: A → B
function composition g ∘ f a set A
(total) function from set A to B

Grp h: G → H
homomorphism composition k ∘ h a group G
group homomorphism from group (G, *) to (H, **)
integers with addition

Ring f: R → S
homomorphism composition g ∘ f a ring R
ring homomorphism from ring R to S
integers with addition + multiplication

Field f: K → H
map composition g ∘ f a field K
linear map
rational numbers i.e. addition + multiplication + division

Top f: A → B
function composition g ∘ f a topological space A

	continuous function from topological space A to B	
Toph	a: X → Y	
function composition b ∘ a		a set X
	a homotopy class i.e. a continuous function f: X → Y mod (homotopic) equivalence	
Vect_k	f: A → B	
map composition g ∘ f		a k-vector space A
	linear map from k-vector space A to B	
R-mod	f: M → N	
homomorphism composition g ∘ f		a left R-module
	homomorphism from R-module M to N	
		Generalization of vector spaces i.e. an abelian group (M, +) with a ring (R, *) as scalars
C^Op	f': Y → X	
		a object X of C
Opposite category	morphism f: X → Y of a category C reversed	
Cat	F: A → B	
functor composition		category A
	functor	

*** Comma category**

Generalized **category of arrows** C^2 by parameterizing the arrows

C^2 = (C ↓ C)	
h: Z → W	object
<f: Z → Z', g: W → W'>: h ⇒ h'	arrow
(F ↓ G)	
<X, Y, h: FX → GY>	object
<f: X → X', g: Y → Y'>: h ⇒ h'	arrow

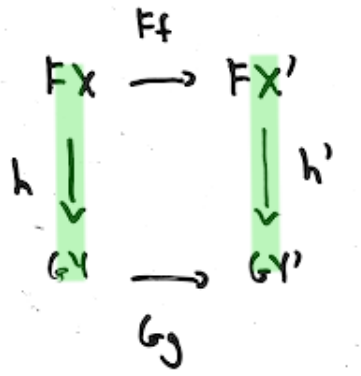
Definition

Let F: A → C, G: B → C

The **comma category (F ↓ G)** is the category of any arrow between F and G

i.e.

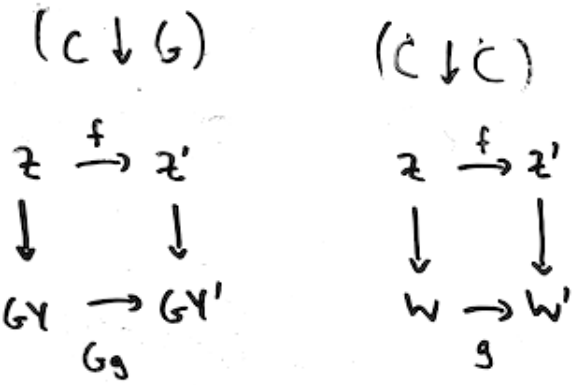
<X ∈ A, Y ∈ B, h: FX → GY>	object i.e. a **triple** representing an arrow
<f: X → X', g: Y → Y'>: <X, Y, h> → <X', Y', h'> where Gg ∘ h = h' ∘ Ff	morphisms i.e. **natural**



Special cases

- Identity F = 1C ≅ C

(C ↓ G)	object <Z ∈ C, Y ∈ B, h: Z → GY>	any Z ∈ C
	arrow <f: Z → Z', g: Y → Y'>	
C^2		
(C ↓ C)	object <Z ∈ C, W ∈ C, h: Z → W>	any pair Z, W ∈ C
	arrow <f: Z → Z', g: W → W'>	

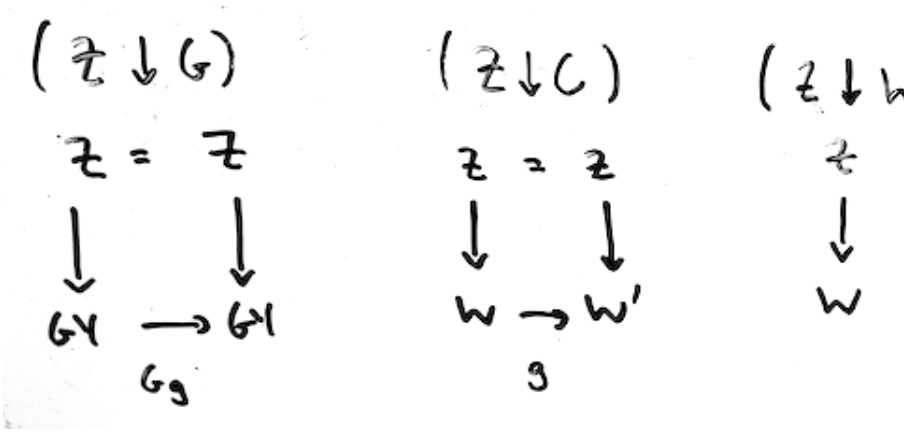


- Fixed point F = ΔX ≅ X ∈ C

(Z ↓ G)	object <Y ∈ B, h: Z → GY>	Z fixed
	arrow g: GY → GY'	
(Z ↓ C)	object <W ∈ C, h: Z →W>	Z fixed
	arrow g: W → W'	

Discrete

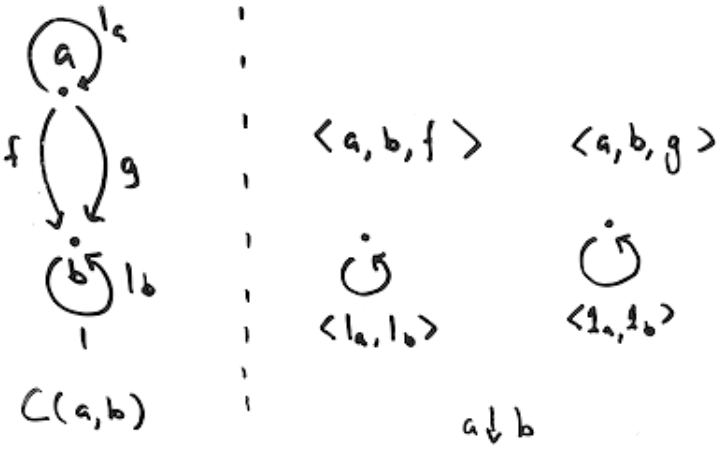
(Z ↓ W)	object h: Z → W	Z, W fixed
	arrow 1	identity only



Examples
- **a ↓ b**: **discrete** category of **a → b**

objects **<a, b, f: a → b>** i.e. any morphism f: a → b

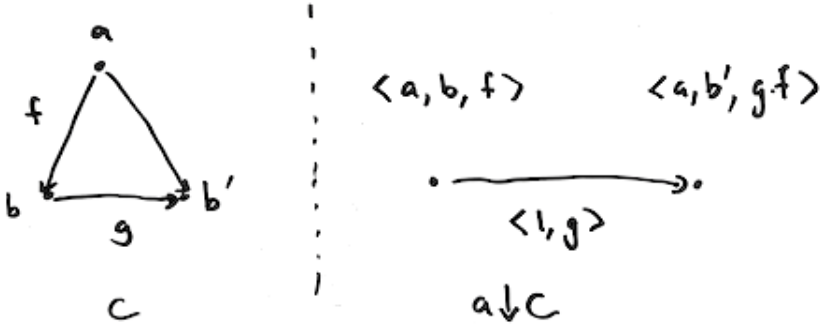
morphisms **<1a, 1b>: <a, b, f> → <a, b, f>** i.e. identity only



- **a ↓ C**: category of any **morphism a → ***

objects **<a, b, f: a → b>** i.e. any morphism f: a → b for any b ∈ C

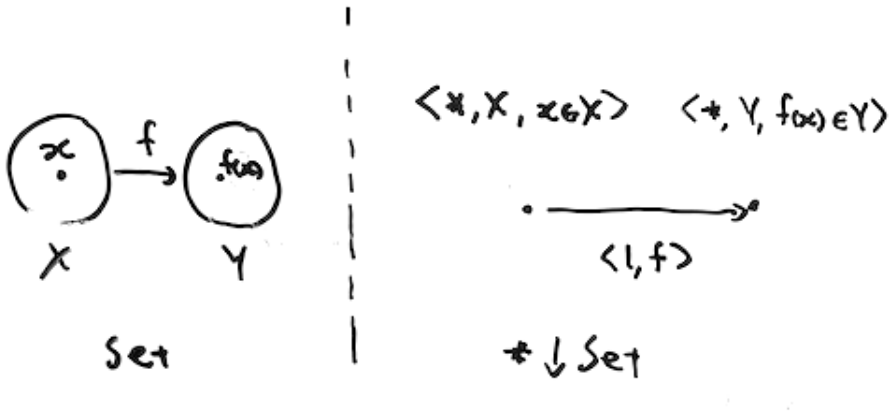
morphisms **<1a, h: b → b'>: <a, b, f> → <a, b', h ∘ f>** i.e. any morphism h: b → b'



- **1 ↓ Set (pointed set)**: category of **elements**

objects **<1, X, x ∈ X>** i.e. an element x ∈ X

morphisms **<1, f: X → Y>**: <1, X, x ∈ X> → <1, Y, f(x) ∈ X> i.e. mapping an element x ∈ X to f(x) ∈ Y

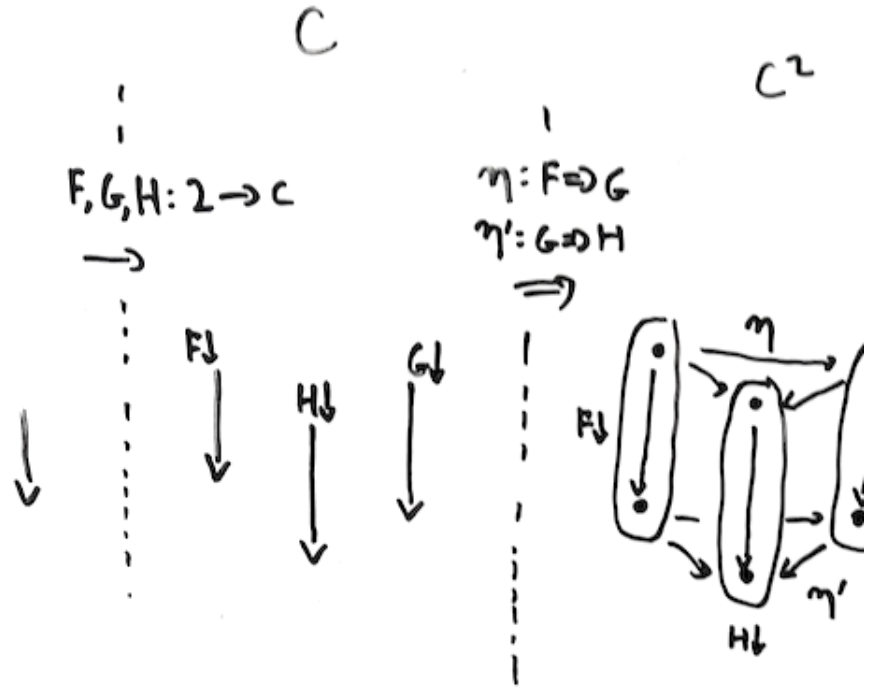


- **(C ↓ C)**: C^2
(C ↓ C) = C^2 but their construction is a dual

C^2 define **morphisms ↓**, then connect top and bottom layers (natural transform)

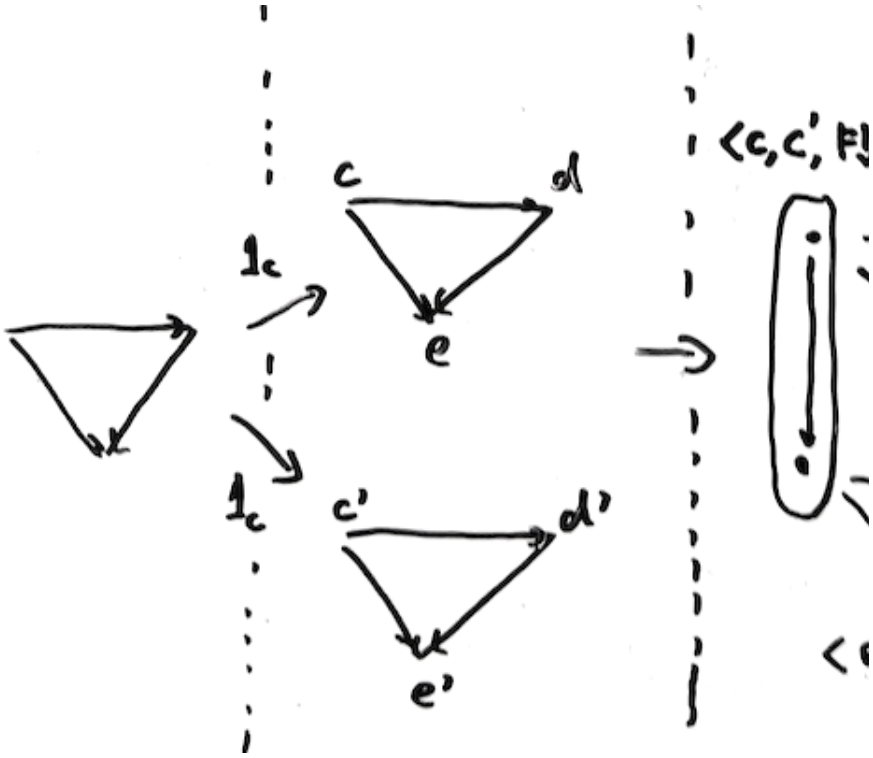
(C ↓ C) define **top and bottom layers (C)**, then join morphisms ↓

C^2



(C ↓ C)

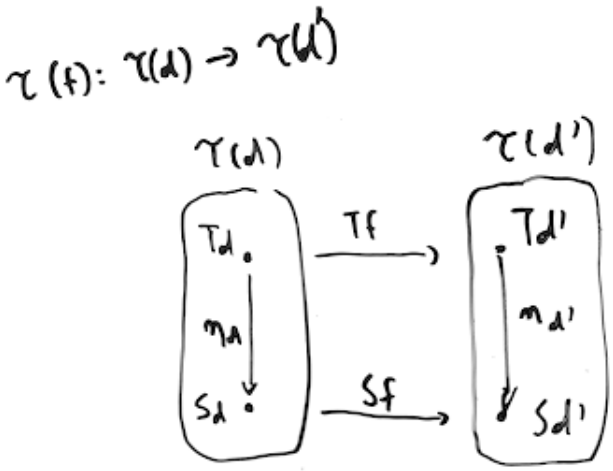
	C^2	(C ↓ C)
objects	F↓	<c, c', F↓>
morphism	η: F ⇒ G	<h,k> = <ηc, ηc'>



- **natural transform** η: F ⇒ F' ≅ **subcategory of** (F ↓ F") :

If η: F ⇒ F' is a natural transform, then η is equivalent to a **subcategory** of

η: F ⇒ F		τ: D → F ↓ F"
ηd : Fd → F'd	<=>	τd = <d, d, ηd>



==========

======

----- Functors -----

Functors

Functor

Wiki: https://en.wikipedia.org/wiki/Functor

A **functor F** is essentially a **homomorphism** of a category
i.e. a mapping F: A → B of morphisms of A and B such that

F(f ∘ g) = Ff ∘ Fg
F(1X) = 1_FX

Definition:
A **functor F** is a map F: A → B from category A to B where for

FX = F(X) ∈ obj(B) X ∈ obj(A)

Ff = F(f): FX → FY ∈ hom(B) f: X → Y ∈ hom(A)

Satisfying:

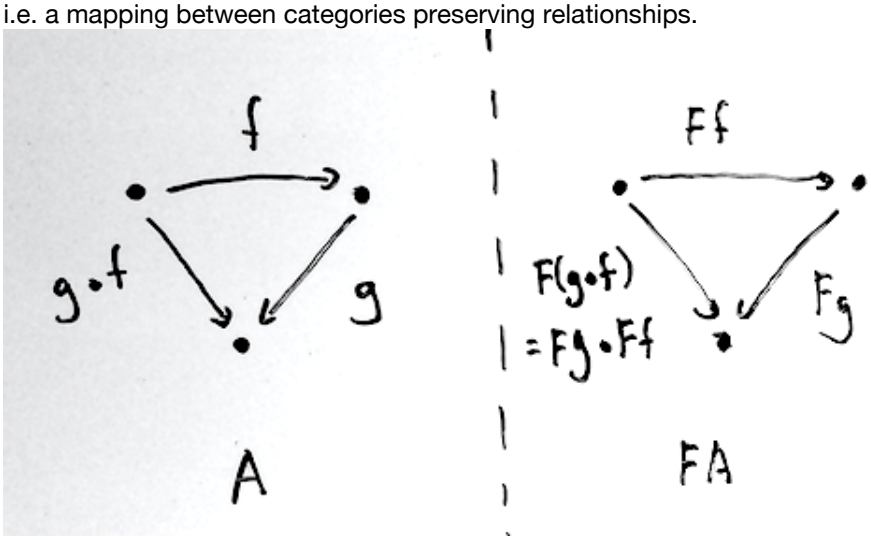
a) **preservation of identity**:

F(1X) = 1F(X)

b) **preservation of composition**:

F(f ∘ g) = Ff ∘ Fg

i.e. A functor is a **homomorphism** of categories
i.e. a mapping between categories preserving relationships.



Lemma: Isomorphism
Functors preserve **isomorphisms**
i.e. if f is an **isomorphism**, then so is Ff and

(Ff)^-1 = F(f^-1)

proof: Ff (Ff)^-1 == F(f f^-1) = F1 = 1. Hence Ff has a right inverse F(f^-1).
Similar for left inverse.

Types of functors

contravariant — Any functor that reverses arrows
i.e.
F: A → B
Ff: FY → FX for any f: X → Y

composition GF: A → C Let F: A → B, G: B →C
Composition

GF: A → C
GFX = G(FX) objects
GFf = G(Ff) morphisms

isomorphism Let F: A → B
injection, surjection

Ff' = Ff <=> f = f

F is **injective aka faithful, 1-1**

any **g: Y → Y' = Ff** for some f: X → X'

F is **surjective**

any **g: FX → FX' = Ff** for some f: X → X'

F is **full**

injective (faithful) + surjective

F is **isomorphic**

F has an **inverse** functor F^-1: B → A

F is **isomorphic**

F^-1F = IA: A → A
FF^-1 = IB: B → B

Proposition: preservation

F, G are **faithful**	=>	FG is **faithful**
F, G are **surjective**	=>	FG is **surjective**
F, G are **isomorphic**	=>	FG is **isomorphic**
FG is **injective** (**faithful**)	=>	G is **injective**
FG is **surjective**	=>	F is **surjective**

Proposition:

φ: I ⇒ F **monic** => F **faithful** i.e. each φX: X → FX is monic/epic

φ: I ⇒ F **epic** + **F faithful** => φ **monic**

ψ: F ⇒ I **epic** => F **faithful**

ψ: I ⇒ F **monic** + **F faithful** => ψ **epic**

*** Common examples**

Identity functor IA: A → A
1:A: A → A
1A(X) = X object map for any object X ∈ A
1A(f) = f arrow mapfor any morphism f: X → Y

Forgetful functor U: **A → B**

A functor mapping to a **weaker** category i.e. just drops some relation

F: Grp → Set	Drop group relation
F: Ring → Abl	Drop multiplication

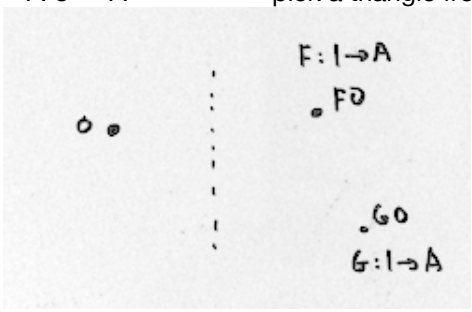
zero functors 0 → A, A → 0

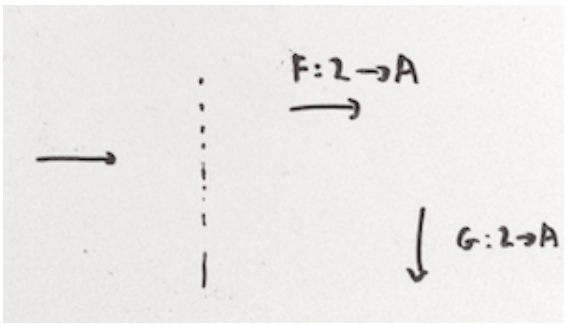
0: 0 → A
no objects/morphisms

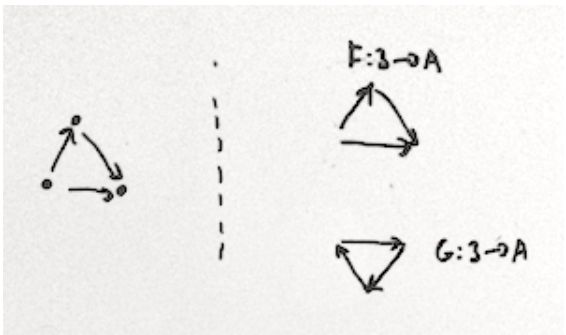
F: A → 0
Undefinable, since
FX = ?

1 → A, 2 → A, 3 → A .. functors

F: 1 → A	pick a point from A
F: 2 → A	pick a morphism from A
F: 3 → A	pick a triangle from A



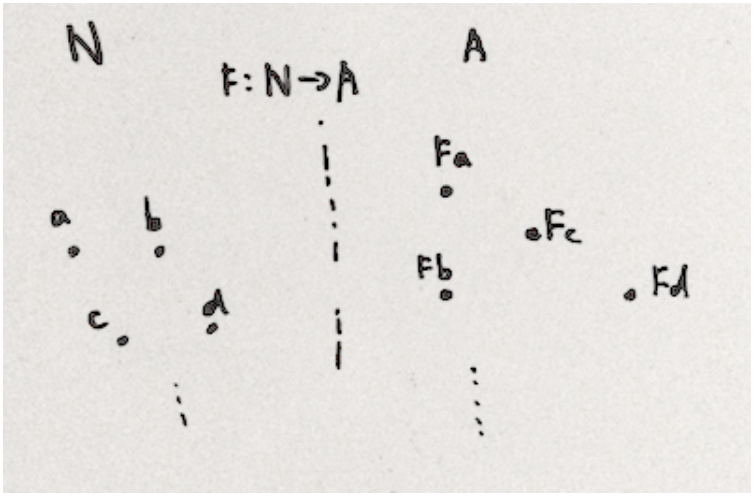




Index functor N → A aka discrete functor
Just a functor from a **discrete category** N

F: N → A

Fi	object map of i ∈ N
-	no arrows



composition of **endofunctors** i.e. any functor F: A → A
If F: A → A, G: A → A

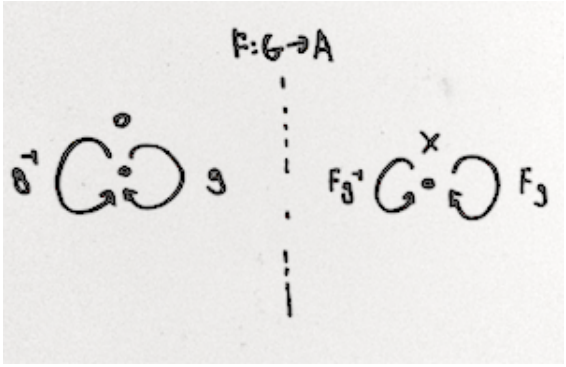
F ∘ G: A → A
(F ∘ G)X = F(GX) object map
(F ∘ G)f = F(Gf) arrow map

e.g.
F ∘ F: A → A
F ∘ F ∘ F: A → A
F ∘ F ∘ F: A → A
F ∘ F ∘ F ∘ F: A → A
...

Group action G → A aka permutation
If G is a group category, F: 0 → A represents a permutations of a fixed set X ∈ A by G

F: G → A
X = F0 ∈ A object map of single object 0 ∈ G
i.e. the permuted object
Fg: X → X arrow map
bijection since Fg is invertible
i.e. Fg ∘ F(g^-1) = 1

e.g.
F: G → Set
X = F0 a set
Fg: X → X permutation



*** Hom set, hom functor**
Wiki: https://en.wikipedia.org/wiki/Hom_functor

Hom-set
A **hom-set of X and Y** is just the set of **all (homo)morphisms X → Y**
i.e. if X is an object of category A

h_X(Y)
Hom(X, Y)
A(X, Y)
{X→Y} = every morphism X → Y
= { f | f: X → Y }

Covariant hom-sets of X
h_X
Hom(X, -)
{X→_} = every morphism from X
= { A(X, Y) | Y ∈ obj(A) }

Contravariant hom-sets of X
h^X
Hom(-, X)
A(-, X)
{_→X} = every morphism to X
= { A(Y, X) | Y ∈ obj(A) }

Hom-functor
A functor induced by **hom-sets.**
Represents a set of mappings with a **fixed base** object
e.g.

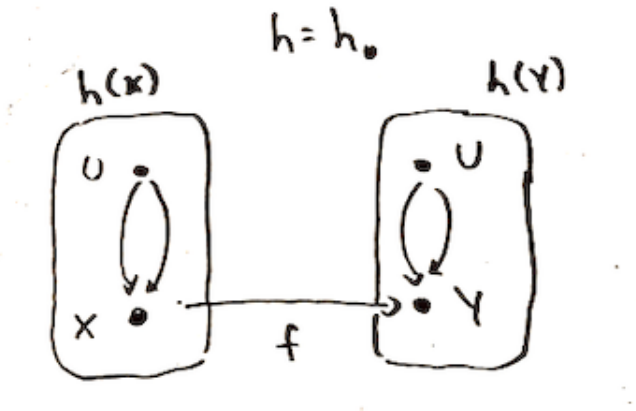
Grp(G, -): Grp → Set
Grp(G, H) = { G → H } group homomorphisms from a base group G

Vect(V, -): Vect → Set
Vect(V, U) = { V → U } linear maps from a base vector space V

Definition:
Let be U an object of category A
The **covariant hom-functor A(U, -): A -> Set** is a functor where the objects are hom-sets and morphisms are composed morphisms

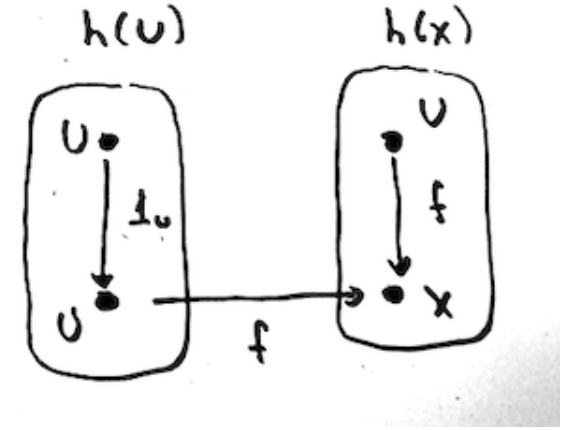
A(U, X) objects (hom-sets)

f*: A(U, X) → A(U, Y) morphisms
f* g = f ∘ g for any morphism f: A(X, Y), g ∈ A(U, X)



Note:
For any f: U → X

f*(1U) = f

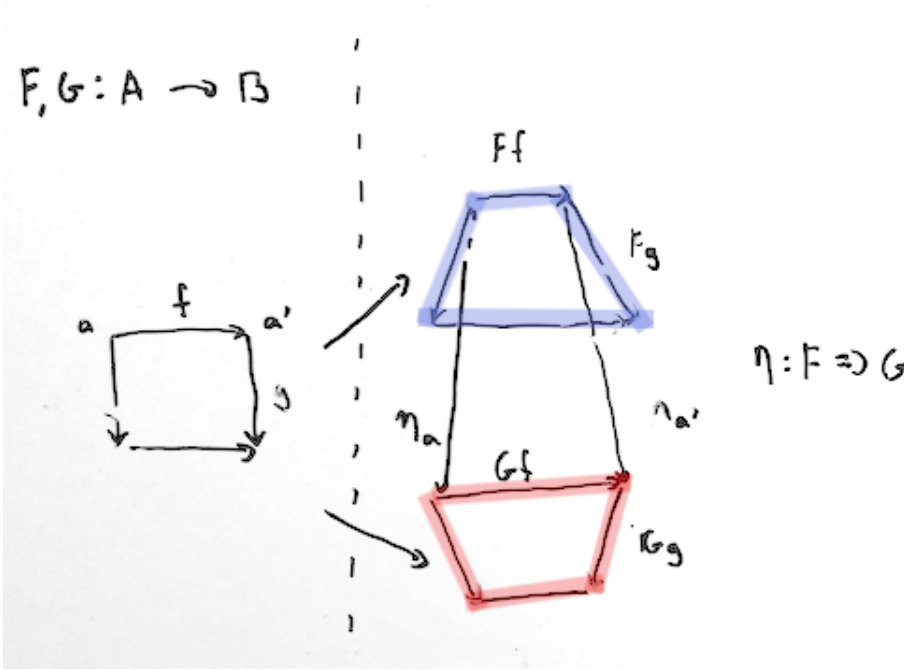


==========

----- Natural transformation -----

Natural transformation
Natural transformation
Wiki: https://en.wikipedia.org/wiki/Natural_transformation
A **natural transformation of F: A → B to G: A → B** is a correspondence of F and G relative to the parametrization by A
i.e. a **transformation η: F ⇒ G** such that

$\eta Y \circ Ff = Gf \circ \eta X$ for any morphism f: X → Y in A



Definition
Let F, G: A → B be functors.
If ηX is a morphism

η**X: FX → GX** any X ∈ obj(A)

such that

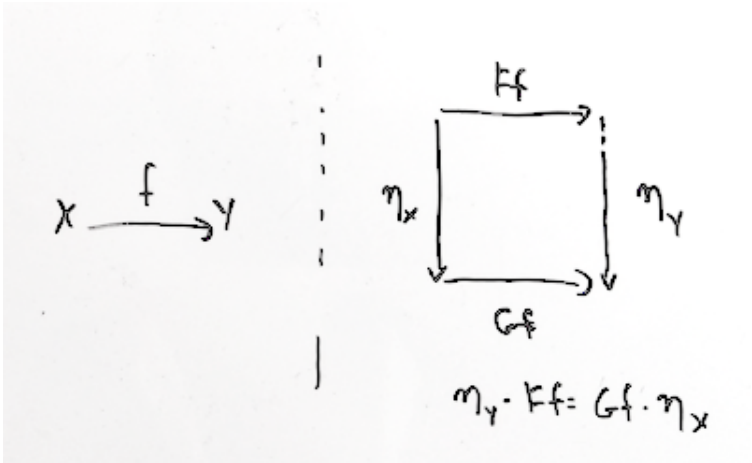
η**Y ∘ Ff = Gf ∘** η**X** any f: X → Y ∈ hom(A)

then ηX is **natural in X**.

η is called a **natural transformation**

η**: F ⇒ G**

with **components** η**X**



Proposition: associativity

Natural transformations preserve associativity of morphisms
i.e.
If f: X → Y, g: Y → Z

(Fg ∘ Ff) ∘ η_X = η_Z ∘ [Gg ∘ Gf]

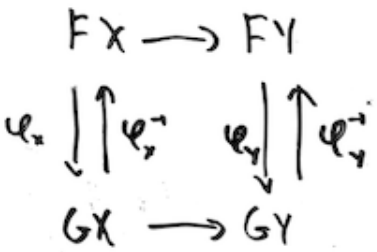
==========
Proof:
[Fg ∘ Ff] ∘ η_X = F(g ∘ f) ∘ η_X
= η_Z ∘ G(g ∘ f)
= η_Z ∘ [Gg ∘ Gf]
QED
==========

Proposition: inverse
If φ: F ⇒ G is a **isomorphic** i.e. each φX: FX → GX is isomorphic,
then the inverses are also natural

φ^-1: G ⇒ F
φ^-1X: GX → FX
φ^-1X = φX^-1

and

φ ∘ φ^-1 = 1: G ⇒ G
φ^-1 ∘ φ = 1: F ⇒ F



==========
Proof
For any f: X → Y

φY ∘ Ff = Gf ∘ φX

=> φY^-1 ∘ (φY ∘ Ff) ∘ φX^-1 = φY^-1 ∘ (Gf ∘ φX)∘ φX^-1
=> Ff ∘ φX^-1 = φY^-1 ∘ Gf
=> φ^-1 is natural

QED
==========

Types of natural transformations

natural isomorphism η: F ⇒ G Each **component** ηX:
FX → GX is an **isomorphism**
η: F ≅ G

vertical composition $\eta' \circ \eta$ **: F ⇒ F''** Composition in the **same category** B

where

F, F': A → B

$\eta' \circ \eta$ F $=\eta\Rightarrow$ F' $=\eta'\Rightarrow$ F''

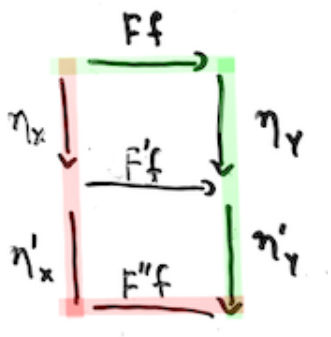
η: F ⇒ F' η': F ⇒ F''

Vertical composition

$\eta' \circ \eta$ **: F ⇒ F''**

$\eta' \circ \eta X$: FX → F''X

$\eta' \circ \eta X = \eta' X \, \eta X$



horizontal composition $\varepsilon * \eta$**: GF ⇒ G'F'** Composition **across multiple categories A → B → C**

where

F, F': A → B G, G': B → C

GF ⇒ G'F/GF' ⇒ G'F'

η: F ⇒ F' ε: G ⇒ G'

Composition $\varepsilon * \eta$

$\varepsilon * \eta$**: GF ⇒ G'F'**

$\varepsilon * \eta$ = ε**F'** ∘ **G**η

as vertical GF ⇒ GF' ⇒ G'F'

= **G'**η ∘ ε**F**

as vertical GF ⇒ G'F ⇒ G'F'

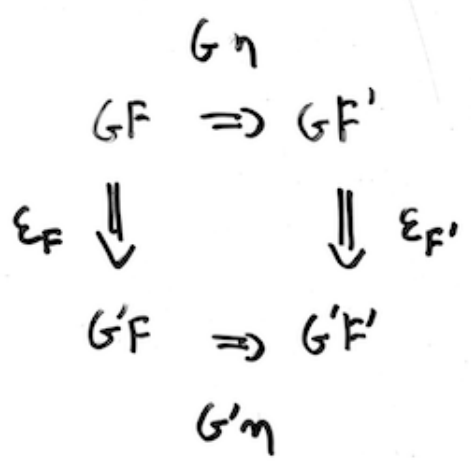
by naturality of ε

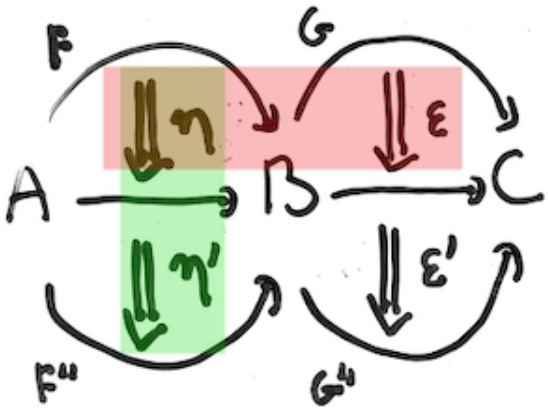
= (ε ∗ 1**F')** ∘

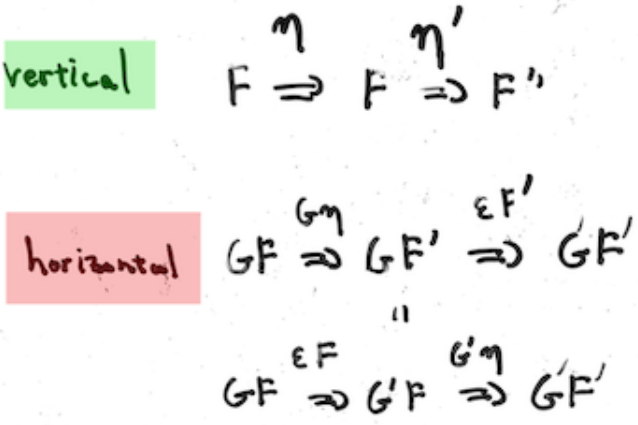
(1G ∗ η) as whiskering

= (1**G'** ∗ η) ∘ **(**ε

∗ **1F)**







whiskering ηK: FK ⇒ F'K

Special case of horizontal composition with 1K: K ⇒ K or 1F: F ⇒ F

$L\eta$: LF ⇒ LF'

where

whiskering Lη

F: A → B K: I → A, L: C → J

Lη: LF ⇒ LF

η: F ⇒ F'

LηX = LηX

whiskering ηK

ηK: FK ⇒ F'K

ηKX = ηFKX

As horizontal

composition with 1

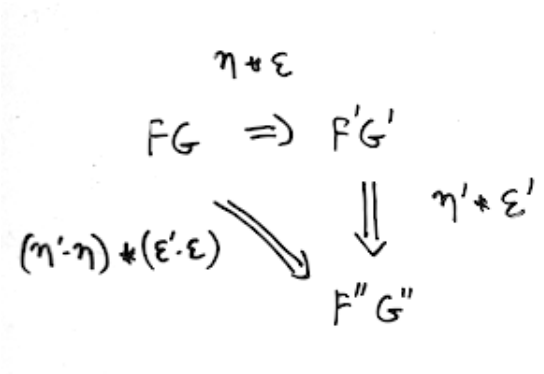
$$\eta\mathbf{K} = \eta * \mathbf{1K}$$

$$\mathbf{L}\eta = \mathbf{1L} * \eta$$

Proposition: interchange
Let
η: F ⇒ F' η': F' ⇒ F''
ε: G ⇒ G' ε': G' ⇒ G''
Then

$$(\eta' \circ \eta) * (\varepsilon' \circ \varepsilon) = (\eta' * \varepsilon') \circ (\eta * \varepsilon)$$

vertical, then horizontal horizontal, then vertical



In particular

$$(\eta' \circ \eta)\mathbf{G} = \eta'\mathbf{G} \circ \eta\mathbf{G} \quad \text{i.e. } (\eta' \circ \eta) * (1G * 1G) = \eta'G \circ \eta G$$

$$\mathbf{K}(\eta' \circ \eta) = \mathbf{K}\eta' \circ \mathbf{K}\eta \quad \text{i.e. } (1K \circ 1K) * (\eta' \circ \eta) = K\eta' \circ K\eta$$

==========

----- Universal property -----

Universal property
Represents the notion of some **canonical object Y0** relative to some object X, often in different categories.
Canonical in the sense that there is a **1-1 correspondence** between their **morphisms**.

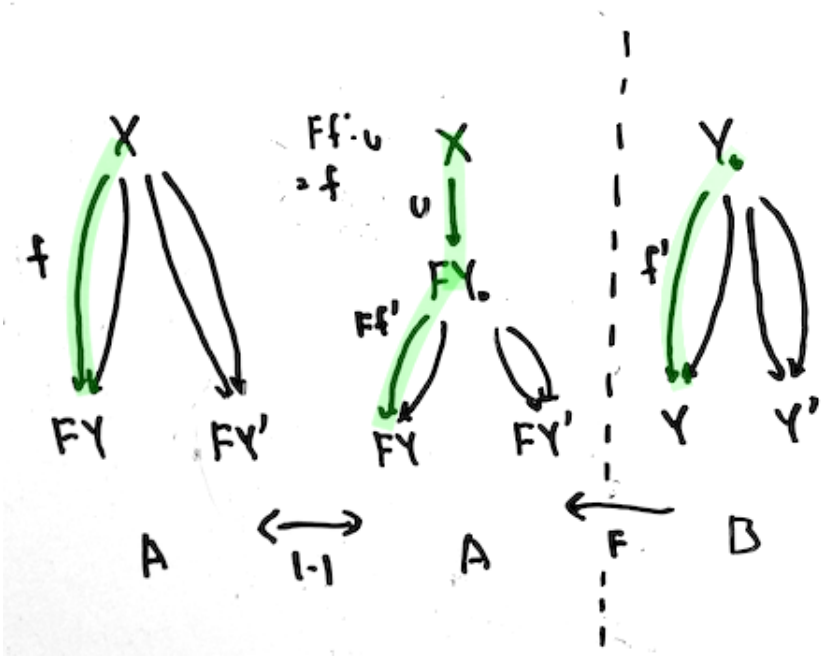
Let X ∈ A, Y0 ∈ B, and F: B → A
The correspondence is provided via a construction of the **universal object Y0**

u: X → FY0 **universal construction** (aka universal morphism) of the object Y0

so that there is a 1-1 correspondence

f: **X → FY** <=> f': **Y0 → Y**
f = Ff' ∘ u

This correspondence is a **universal property**.



Universal property can also be seen as an analogue of a **line ax + b**.

Line y = ax + b: X → Y <=>	**Universal property (arrow u)** {Y0 → _} ≅ {X0 → F_} <=> **Universal property (element x)** {Y0 → _} ≅ F_
base b ∈ Y	base u: X0 → FY0
	base x ∈ FY0
factor a: X → Y	functor F: A → B
linear a(x + x') = ax + ax'	linear F(f ∘ f') = Ff ∘ Ff'
X **Y**	**{Y0 → _}** **{X0 → F_}** **{Y0 → _}** **F_**
x ∈ X <=> y ∈ Y in B	f': Y0 → Y in A <=> f: X0 → FY f:Y0 → Y in A <=> y ∈ FY
ax + b = y	**Ff' ∘ u = f** **Ff'(x) = y**

Terminology

e.g.

Product XxY is a universal property	
Y0 = XxY	product XxY is a universal object
u: (X, Y) → FY0 = (XxY, XxY)	universal construction of the product XxY

e.g.

Free vector space VX is a universal property	
Y0 = VX (free vector space)	Free vector space VX is a universal object
u: X → FY0 = UVX	universal construction of free VX

Proposition: Category <-> Set correspondence
Let F: B → A
Equivalent statements:

a) **universal morphism** u: X → FY0 from X to F

f: X → FY <=> f': Y0 → Y
f = Ff' ∘ u

b) u: X → FY0 is a **universal element** in {X → F_}

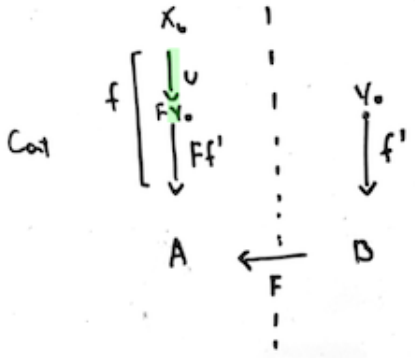
f ∈ {X → FY} <=> f': Y0 → Y
f = Ff'*(u)
= Ff' ∘ u

c) bijective φ: {Y0→_} ⇒ {X→F_} with **generator** φY0(1Y0) = u: X → FY0

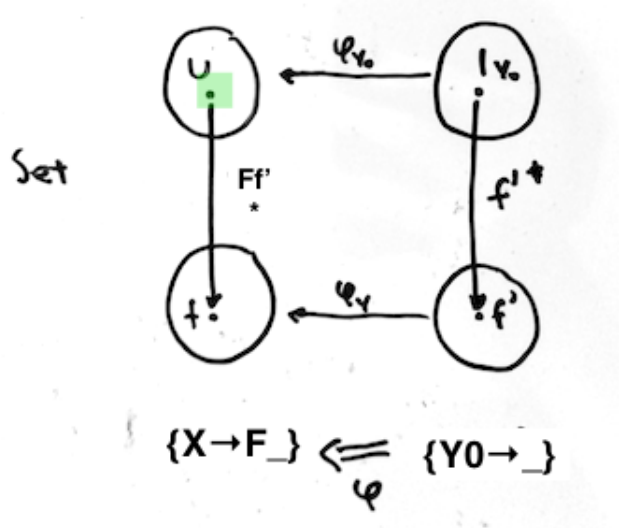
f ∈ {X→ FY} <=> f' ∈ {Y0→ Y}
f = φY(f')
= Ff'*(u)
= Ff' ∘ u

d) **initial element** u: X → FY0 of X ↓ F

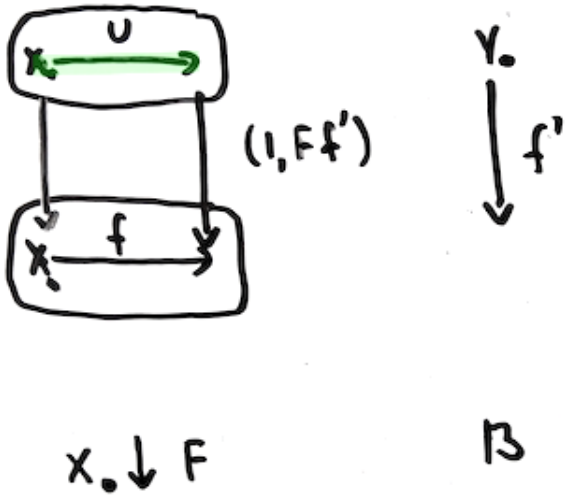
As categories a)



As sets b), c)

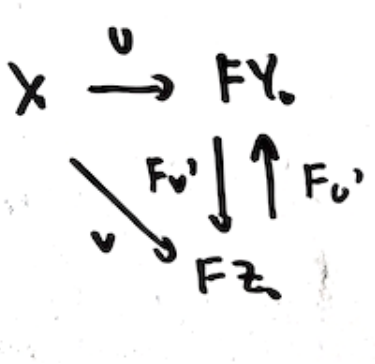


As comma d)



Isomorphism theorem
Let F: B → A
If both u: X → FY0 and v: X → FZ0 are both universal arrows (i.e. universal properties of the **same X and F**), then they are isomorphic

Y0 ≅ Z0



==========
Proof

u	=	Fu' ∘ v	u in terms of universal v
	=	Fu' ∘ (Fv' ∘ u)	v in terms of universal u
	=	F(u' ∘ v') ∘ u	

but also

u	=	F1Y0 ∘ u	trivial

=> 1Y0 = u'v' by uniqueness of universal property of u

Similarly v'u' = 1Z0.
QED
==========

Proposition: isomorphism preservation
Let F: B → A be a universal property with universal arrow u: X → FY0.
Then

universal property **preserves isomorphisms** <=>
universal arrow is **isomorphic**
i.e. f': Y0 → Y isomorphism
=> f: X → FY isomorphism

==========

Proof:
- universal property preserves isomorphism

u = F1 ∘ u isomorphic since 1 is isomorphic
=> u is an isomorphism

- u isomorphism
Then

f': Y0 → Y isomorphism
=> f = Ff' ∘ u isomorphism since Ff' and u are both isomorphic

QED
==========

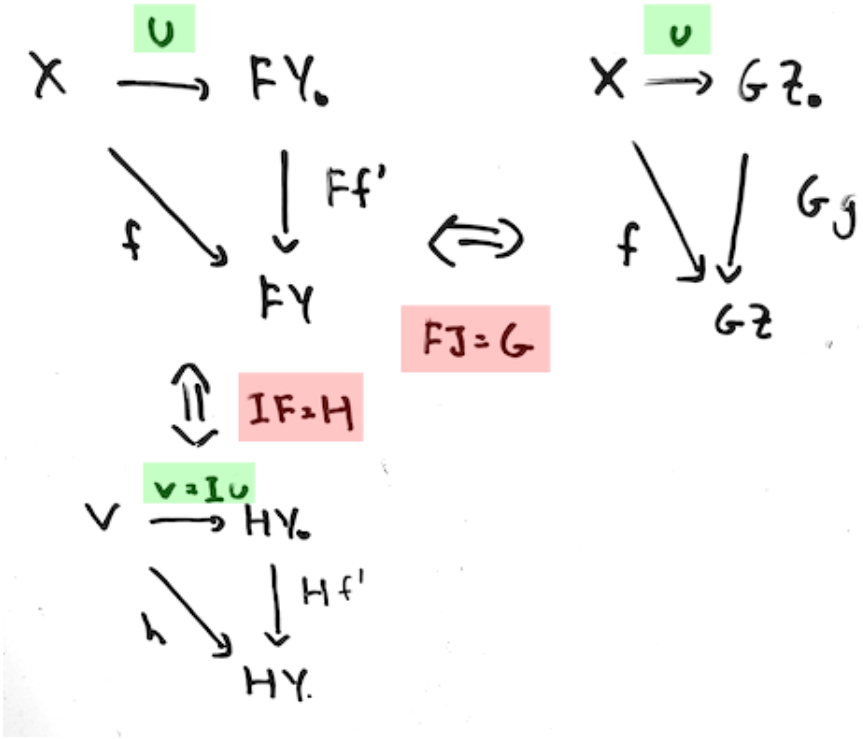
Proposition: functor isomorphism
Let F: B → A be a universal property with universal arrow u: X → FY0.
Then universal property is **preserved** by **functor isomorphisms** C → B and A → D

- **isomorphism J: C → B**
If
G: C → A
G = FJ
then G is a universal property

universal arrow		
u: X → FY0	<=>	**u: X → GZ0**
	Y0 = JZ0	
f: X → FY		f: X → GZ
f = Ff' ∘ u	<=>	**f = Gg' ∘ u**
unique f': Y0 → Y	Y = JZ	unique g': Z0 → Z
	f' = Jg'	

- **isomorphism I: A → D**
Let
H: B → D
H = IF
Then H is a universal property

universal		
u: X → FY0	<=>	**v: V → HY0**
	Iu = v	
	IX = V	
f: X → FY		h: V → HY
f = Ff' ∘ u	<=>	**h = Hf' ∘ v**
unique f': Y0 → Y	IX = V	unique f': Y0 → Y
	If = h	



==========
Proof

1) J: C → B

f	=	Ff' ∘ u	
	=	FJg' ∘ u	Jg' = f'
	=	Gg' ∘ u	FJ = G

=> by isomorphism

f = Ff' ∘ u <=> f = Gg' ∘ u

2) I: A → D

h	=	If	
	=	I (Ff' ∘ u)	by universality
	=	IFf' ∘ Iu	
	=	Hf' ∘ v	

=> since I is isomorphic,

f = Ff' ∘ u	<=>	h = Hf' ∘ v
	If = h	
	Iu = v	

QED
==========

Examples

- Vect: **basis map** X → W extended to linear map f: V(X) → W

U: Vect → Set	forgetful functor
X ∈ Set	a basis

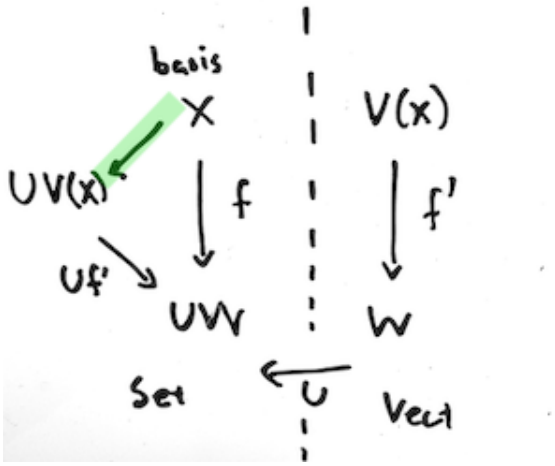
universal property

V(X)	universal object = **free vector space containing X**
i: X → V(X)	universal construction = **injection**
i(x) = x	

property: A basis map uniquely extends to a linear map

f = Uf' ∘ i
f(x) = f'(x) for any base vector x ∈ X

basis map	<==>	**linear map**
f: X → UW		f': V(X) → W



- Free group: **symbol map** X→ G used to collapse the free group FX into G

U: Grp → Set	forgetful functor
X ∈ Set	a set of symbols

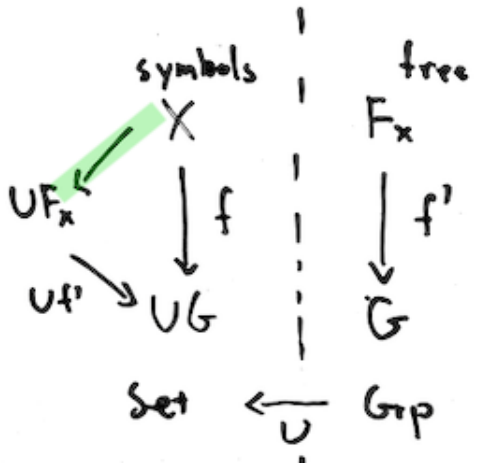
universal property

FX	universal object = **free group** i.e. a canonical group containing X
i: X → FX	universal construction = **injection**
i(x)	

property: A symbol map uniquely generates a homomorphism

f = Uf' ∘ i
f(a) = f'(a) for any symbol a ∈ X

symbol map	<==>	**homomorphism**
f: X → UG		f': FX → G
		f'(x) = f(x)



- **disjoint union X⊎Y** (aka **direct sum X⊕Y**) : two component maps X → Z, Y → Z as a **single map X⊎Y → Z**
  cartesian product X x Y (aka **direct product X⊗Y**): two component maps Z → X, Z → Y as a **single map Z → X x Y**

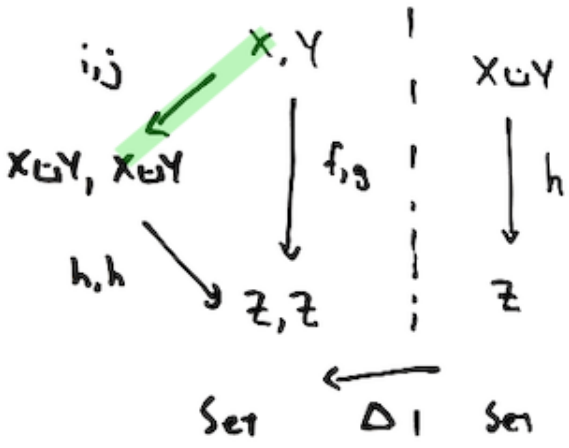
universal property

X⊎Y	universal object = **disjoint union** of two objects as a single object
i: X → X⊎Y	universal construction = injections
j: Y → X⊎Y	

property: Any two unique component maps f,g can be represented as a unique single map h

(f, g) = (h, h) ∘ (i, j)

component maps	<=>	**single map**
f: X → Z		h: X⊎Y → Z
g: Y → Z		h(x) = f(x)
		h(y) = g(y)



Dually

universal property

X x Y — universal object = **direct product** of two objects as a single object

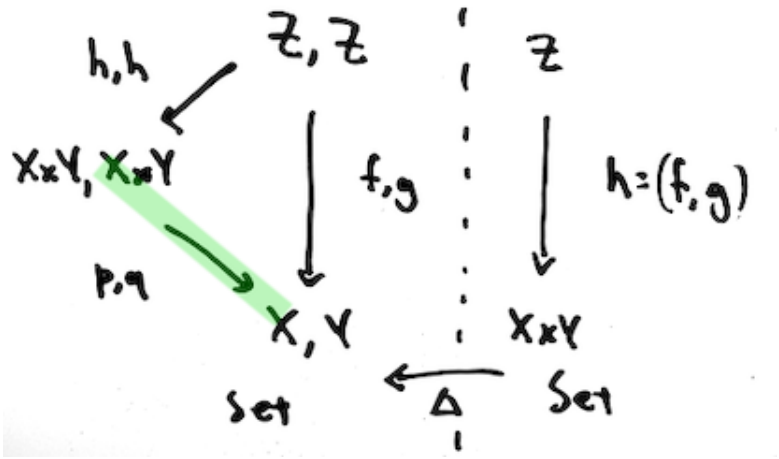
p: X x Y → X — universal construction = projections

q: X x Y → Y

property:

$$(f, g) = (p, q) \circ (h, h)$$

component maps	<=>	**single map**
f: Z → X		h: Z → X x Y
g: Z → Y		h(v) = (f(z), g(z))



==========

----- Yoneda Lemma -----

Wiki: https://en.wikipedia.org/wiki/Yoneda_lemma

Hom functor

Morphism map of {X→_}

Composition

h*: {X→Y} → {X→Y'} — for h: Y → Y'

h*f = h ∘ f

*g: {X→Y} → {X'→Y} — for g: X' → X

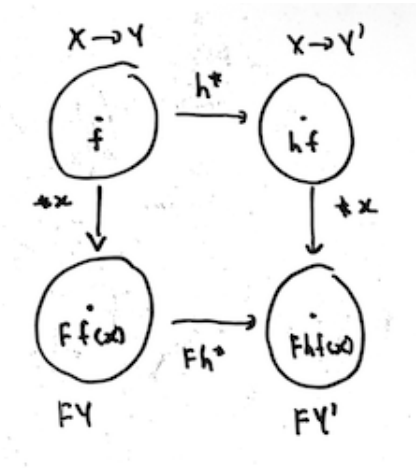
*gf = f ∘ g

Dual is evaluation for F: A → Set

Evaluation

*x: {X→Y} → FY — for x ∈ FX

*xf = Ff(x)



Composition can be interpreted as a special case of evaluation

Proposition
Let
F: A → {X→_}
Fh = h* for h: Y → Y'
G: A^op → {_→Y}
Gf^op = *g for g: X' → X

Then as evaluation

*x: {X→Y} → FY for x = g ∈ FX = {X→X}
*x = *g
*xf = *g(f) = f ∘ g

*y: {X→Y} → GX for y = h ∈ GY = {Y→Y}
y = h
yf = h(f) = h ∘ f

Proposition: natural evaluation
Let F: A → Set
Then *x is natural

*x: {X→_} ⇒ F
*xY: {X→Y} → FY
*xYf = Ff(x) ∈ FY

==========
For any f: Y → Y' and g ∈ {X→Y}

(*xY' ∘ f*) g = *x (fg)
= Ffg (x)

(Ff ∘ *xX) g = Ff (g(x))
= Ffg (x)

=> *x is natural
QED

==========

Yoneda Lemma

Fundamental concepts of category theory are of the form **{X→ _} ⇒ F** for some **set valued** F: A → Set
In particular, embeddings of the form **{X → _} ⇒ {Y → G_}**
i.e. a **1-1 correspondence i.e. natural transformation**, mapping of morphisms.

Yoneda Lemma provides a way of **computing** such φ
e.g.

Universal property

φ: {Y0 →_} ⇒ {X → F_}	isomorphism
φf = Ff ∘ u	u = φ1Y0 ∈ FY0 **universal element** of F

Adjunction

φ: {FX →_} ⇒ {X → G_}	isomorphism
φf = Gf ∘ ηX	ηX = φ1FX **unit**
-φ: {_ → GY} ⇒ {F_ → Y}	
-φg = εY ∘ Ff	εY = -φ1GY **counit**

Proposition: evaluation
Let F: A → Set
A natural transformation {X0→_} ⇒ F is an **evaluation** *x for some x ∈ FX0.
i.e.

φ : {X0→_} ⇒ F <=> x ∈ FX0
φ = *x
φX0(1X0) = x

Hence, for any f ∈ {X0 → X}

φX(f) ∈ FX
φ**X(f)** = Ff*(x)
= **Ff(x)**

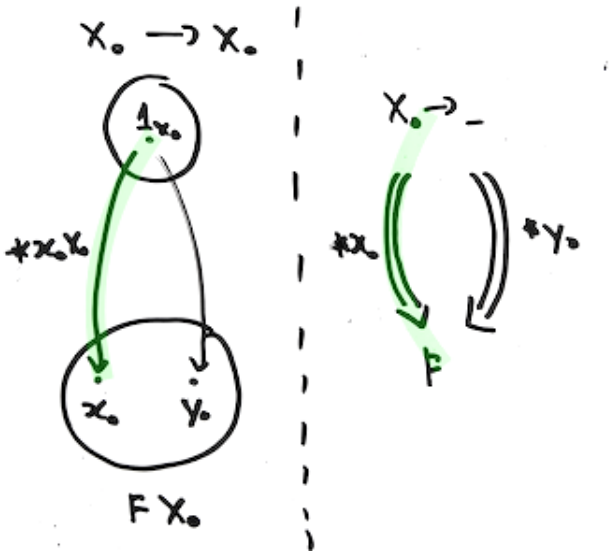
Corollary 1: Universal element
If *X0 is the X0 application functor ***X0 F = FX0**

***X0: Set^A → Set**

***X0**F = FX0	object map	any functor F: A → Set
***X0**φ: FX0 → F'X0		
***X0**φ = φX0	morphism map	any natural φ: F ⇒ F'

then **1=1X0** is a universal element.
i.e. for any x0 ∈ FX0

*X0*x0 (1) = *x0X0(1) = x0



Yoneda is just a restatement of the proposition

Corollary 2: Yoneda Lemma
Let F: A → Set.
Then there is a 1-1 correspondence between the sets

$\{ \varphi : \{X0 \to _\} \Rightarrow F \}$ <=> FX0

φ**1X0** ∈ **FX0**

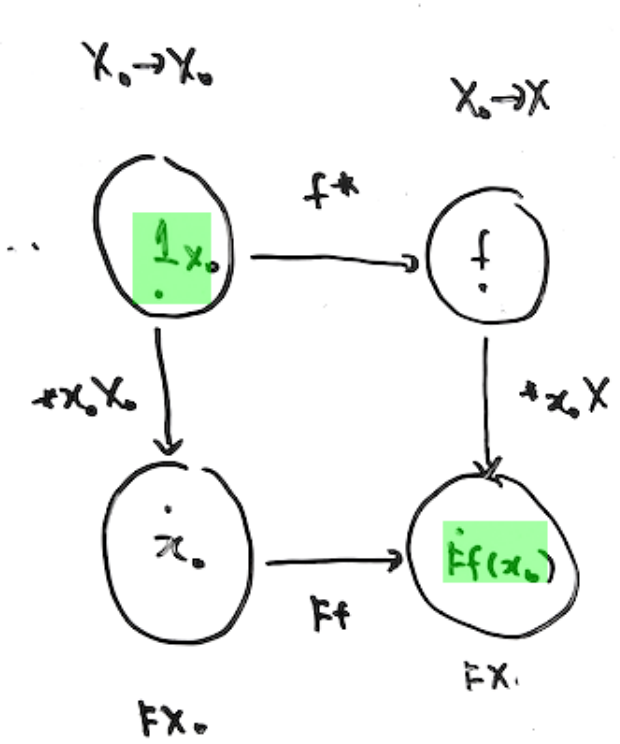
==========
Proposition Proof:
=>) Let

φ: {X0→_} ⇒ F natural
x0 = φX0 (1X0)

Then for any f: X0 → X

$\varphi X(f) = \varphi X(f \circ 1X0)$
$= (Ff \circ \varphi X0)\ (1X0)$ by definition of natural
$= Ff\ (\varphi X0(1X0))$
$= Ff\ (x0)$

=> φ = *x0 where *x0 = φX0(1X0) ∈ FX0



<=) *x is natural

QED

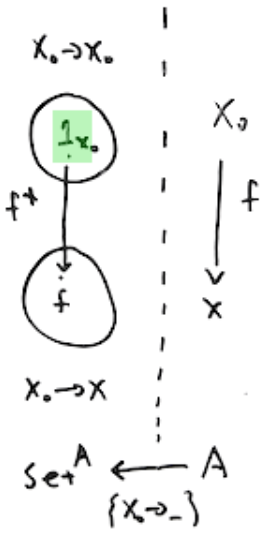
==========

Universality

Proposition: universal property of {X0→_}

1X0 ∈ {X0→X0} is a universal element of {X0→_}: A → Set with

A		**Set**
f: X0 → X	<=>	f ∈ {X0 → X}
	f = f*(1X0)	
	= f ∘ 1X0	

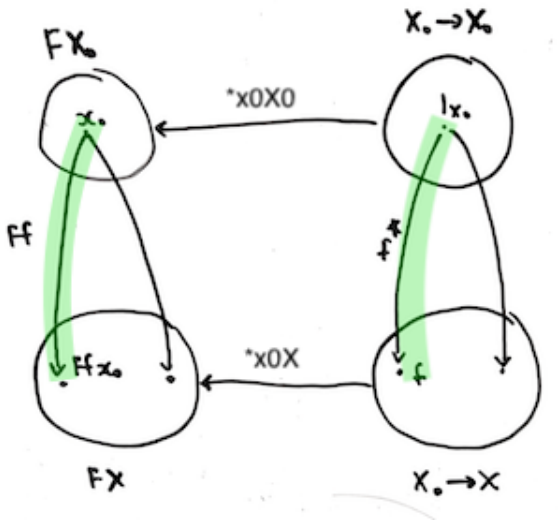


This suggests that a map x0*: {X0→_} ⇒ F and 1X0 **induces** a universal element for F too.

Representation
If φ: {X0→_} ⇒ T is **isomorphic** (i.e. each φX is invertible), then φ is called a **representation (of F).**

Corollary 2: universal element
Let F: A → S

[**universal property**]	<=>	[**representation**]
x0 ∈ FX0 is a universal element in F		*x0: {X0→_} ⇒ F is a representation
	<=>	[**Yoneda embedding**]
		*x0: {X0→_} ⇒ F is a terminal element in {_→_} ↓ 1F



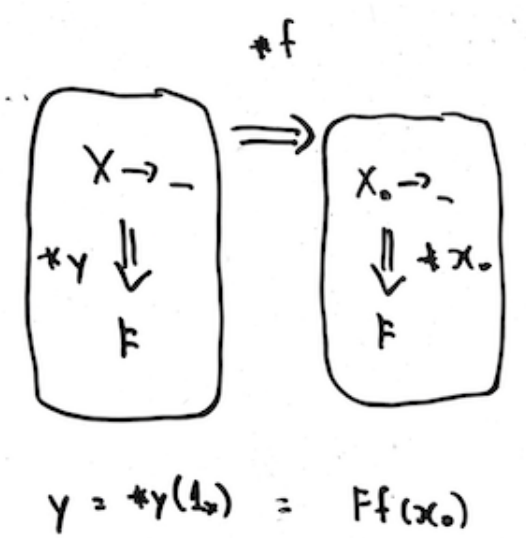
Note:

1X0 ∈ {X0→X0}	universal element of {X0→_} Also, a special case with T = {X0→_}
*x0 X0(1X0) = x0 ∈ FX0	universal element of F

==========
Proof
- **terminal element <=> representation**
Idea: a map of the domains {X→_} ⇒ {X0→_} of {_→_} ↓ 1_F is a Yoneda embedding.

*x0 is a representation	<=>	∃! f y = Ff(x0) for any y ∈ FX
	<=>	∃! **embedding** *f: {X→_} ⇒ {X0→_} with *f ∘ *y = *x0 for any y ∈ FX

<=> x0* is a terminal element



QED

==========

Yoneda embedding

Special case where F is also a hom functor

Yoneda embedding

{X0→_} ⇒ {X1→_}

Yoneda embedding functor

{_→_}: A^op → Set^A

{_→_} X = {X→_} object map

{_→_} f = *f: {X→_} ⇒ {Y→_} morphism map

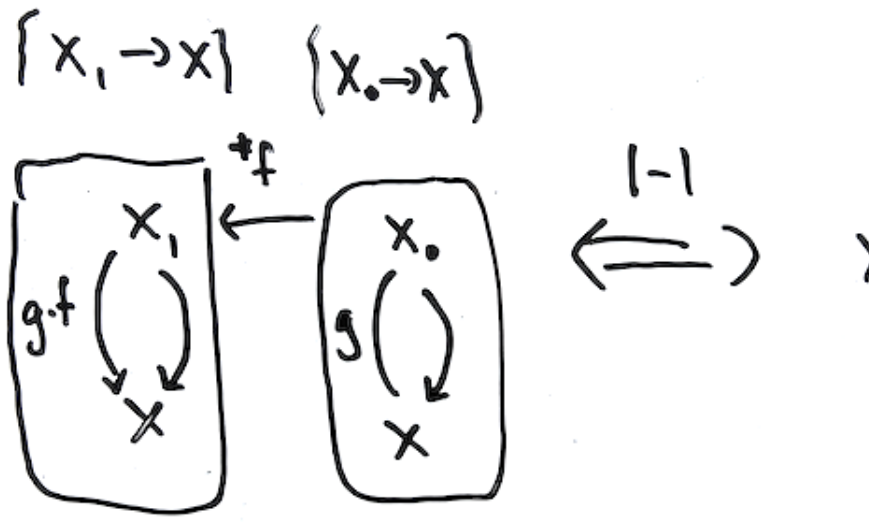
for f: Y → X by Yoneda

Proposition: Yoneda lemma

There is a 1-1 correspondence

({X0→_} ⇒ {X1→_})		**{X1→X0}**
*f: {X0→_} ⇒ {X1→_}	<=>	f: X1 → X0

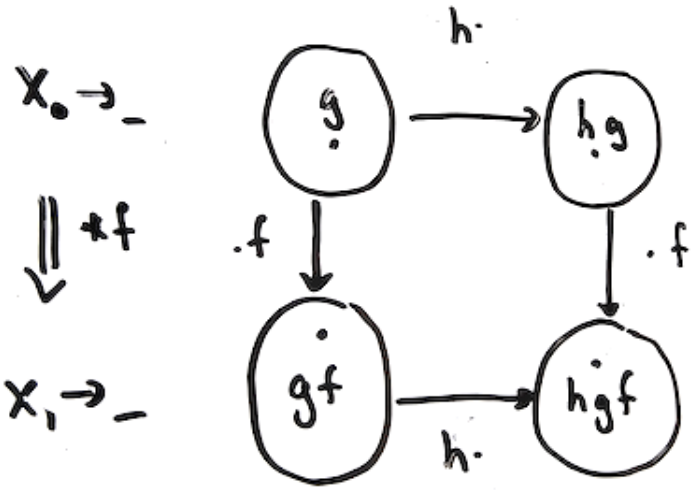
i.e. the Yoneda embedding functor is **full** and **faithful**



Corollary: commutation
Commutations are all function compositions

h: X → X'
h*: {X0→X} → {X0→X'}
h* = (h ∘)

f: X1 → X0
*fX: {X0→X} → {X1→X}
*fX **= (∘ f)**



==========
Proof
- morphism map is bijective:

full
Any {X→_} ⇒ {Y→_} is of the form *f for some f: Y → X
=> {_→_} f = *f

faithful
*f = {_→_} f = {_→_} g = *g for some f, g: Y → X
=> f = g

QED
==========

Proposition: Isomorphism

{X0→_} ≅ {X1→_} <==> X0 ≅ X1

with **isomorphisms**

*f: {X0→_} ⇒ {X1→_}	**<=>**	f: X0 → X1
(*f)^-1 ∘ *f = I	**(*f)^-1 = *(f^-1)**	f^-1 ∘ f = 1X0
*f ∘ (*f)^-1 = I		f ∘ f^-1 = 1X1

Proof: by 1-1 correspondence of f and *f. QED

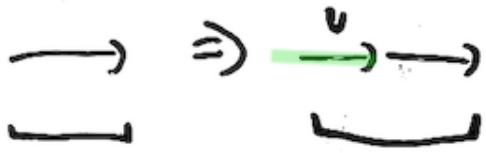
==========

----- Limit/Colimit -----

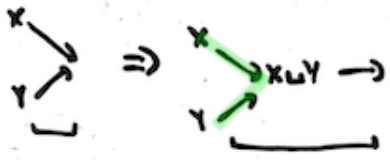
Limits

Idea
Limits and colimits are **universal properties** of arrows

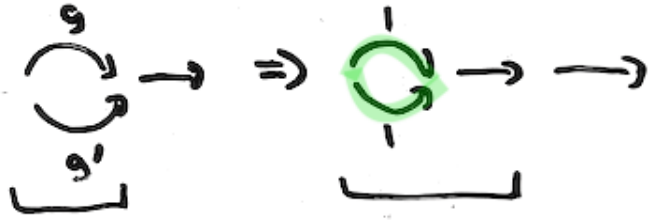
Basic universal property universal**: u**



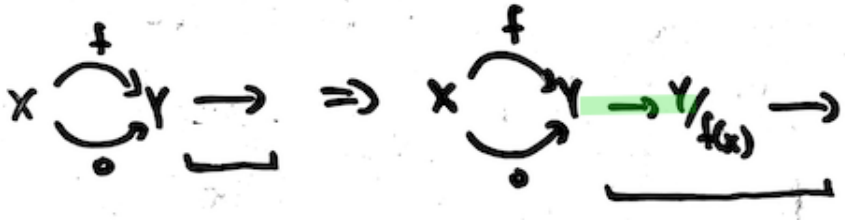
Coproduct universal: injections p: X → X ⊔ Y, q: Y → X ⊔ Y



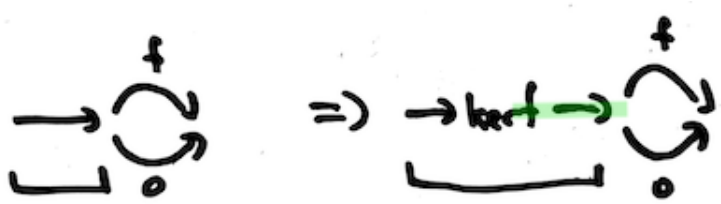
Monic universal: (1, 1) i.e. g = g'



Quotient universal: quotient q: Y → Y/f(X)



Kernel universal: kernel i: ker f → X

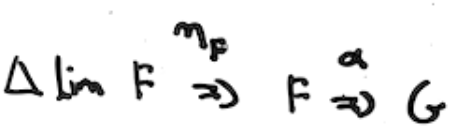


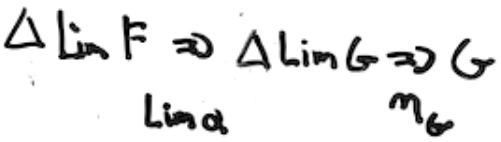
Proposition: naturality
Limits are natural, if they exist.
i.e. if Lim F exists for all F ∈ B^A, then

Limit functor
Lim: A^J → J
Lim α: Lim F → Lim G α: F ⇒ G

Colimit functor
Colim: A^J → J
Colim α: Colim F → Colim G α: F ⇒ G

naturality
Lim (β ・ α) = Lim β ∘ Lim α for any α: F ⇒ G
Colim (β ・ α) = Colim β ∘ Colim α β: G ⇒ H





==========
Proof
- **Lim α definition**
Let α: F ⇒ G. Then

α ・ ηF: **Δ**Lim F ⇒ G

=> there is a unique σ: Lim F → Lim G

α ・ ηF = ηG ・ **Δ**σ
= ηG ・ **Δ**Lim α define Lim α := σ

- **naturality**

β α: Lim F ⇒ H

=>

$(\beta\alpha)\ \eta F \quad = \quad \eta H\ \Delta \text{Lim}\ \beta\alpha \quad$ by definition of Lim

$(\beta\alpha)\ \eta F \quad = \quad \beta\ (\eta G\ \Delta \text{Lim}\ \alpha)$
$\quad = \quad \eta H\ \Delta \text{Lim}\ \beta\ \Delta \text{Lim}\ \alpha$

=> Lim $\beta\alpha$ = Lim β Lim α by uniqueness

QED
==========

==========
======
----- Product, coproduct -----

Product X ⊓ Y (aka cartesian product X x Y, direct product X⊗Y)
Direct product X x Y for categories
i.e. a category with a way to represent **two objects** X, Y as a **single object** X x Y
Hence **two morphisms f: Z → X, g: Z → Y** as a **single morphism [f,g]: Z → X x Y**

Definiton
If it exists, X x Y is an object with **projections**

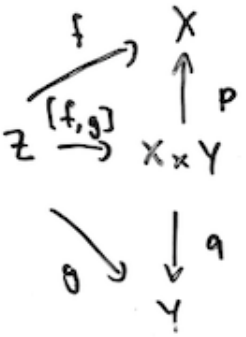
p: X x Y → X q: X x Y → Y

such that there is a **1-1** correspondence

f: Z → X	<=>	h: Z → X x Y
g: Z → Y	**f = p ∘ h**	
	g = q ∘ h	

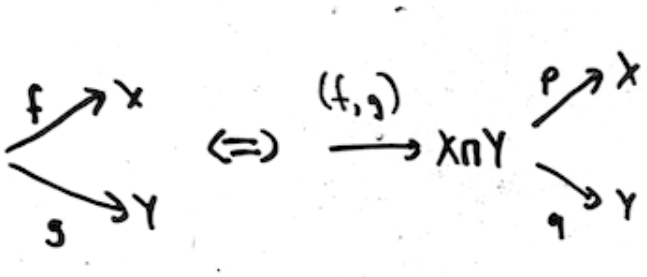
Define

[f, g]: Z → X x Y
f = p ∘ [f, g]
g = q ∘ [f, g]



Universal property

Two arrows		Single arrow
f: Z → X	<=>	[f,g]: Z → X x Y where
g: Z → Y		f = p ∘ [f,g] g = q ∘ [f,g]



Proposition: bifunctor
Product is a **bifunctor**
i.e. If A is a category with products, then x is a bifunctor:

x: A x A → A
s x t: X x Y → X' x Y' where s: X → X', t: Y → Y'
s x t = [s ∘ pX, t ∘ pY]

Proposition: arrows
Let
f: Z → X g: Z → Y
s: X → X' t: Y → Y'

- Arrows

Any h: Z → X x Y
h: Z → X x Y
h = [f, g]

p ∘ [f, g] = p ∘ h
= f
q ∘ [f, g] = q ∘ h
= g

Special case: bifunctor
s x t: X x Y → X' x Y'
s x t = [s ∘ p, t ∘ q]

p' ∘ s x t = p' ∘ [s ∘ p, t ∘ q]
= s ∘ p

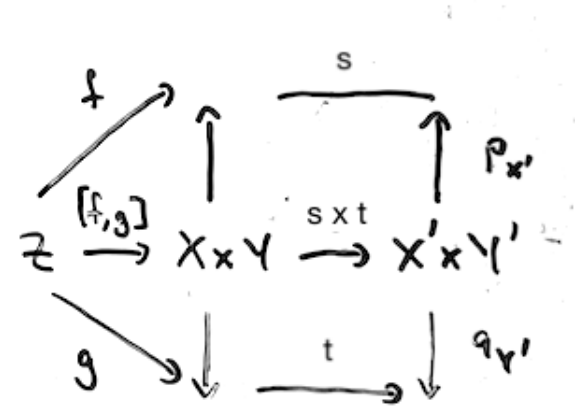
q' ∘ s x t = q' ∘ [s ∘ p, t ∘ q]
= t ∘ q

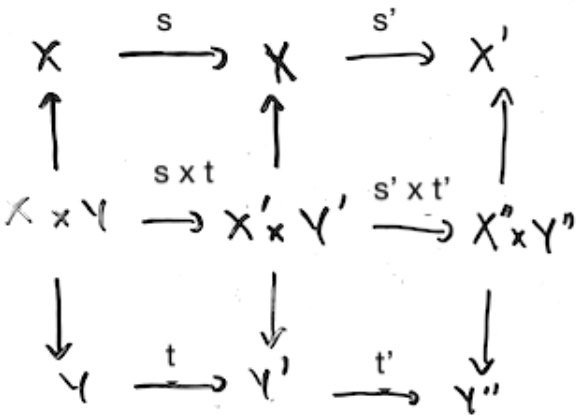
- Compositions:

Basic
[f, g] ∘ h = [f ∘ h, g ∘ h]

(s x t) ∘ [f, g] = [s ∘ p ∘ [f, g], t ∘ q ∘ [f, g]]
= [s ∘ f, t ∘ g]

Bifunctor
(s' x t') ∘ (s x t) = [s' ∘ p' ∘ (s' x t'), t' ∘ q' ∘ (s' x t')]
= [s' ∘ s ∘ p, t' ∘ t ∘ q]
= (s' ∘ s) x (t' ∘ t)





==========

Proof
- composition:
By the diagram

==========

Coproduct X ⊔ Y (aka disjoint union X⊎Y, direct sum X⊕Y)
Disjoint union X⊎Y for categories
Dual of product.

Definition
If it exists, then X ⊔ Y is an object with **injections**

i: X → X ⊔ Y j: Y → X ⊔ Y

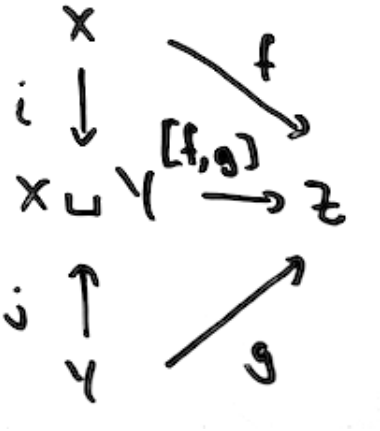
such that there is a 1-1 correspondence

f: X → Z	<=>	h: X ⊔ Y → Z
g: Y → Z	**f = h ∘ i**	
	g = h ∘ j	

Define

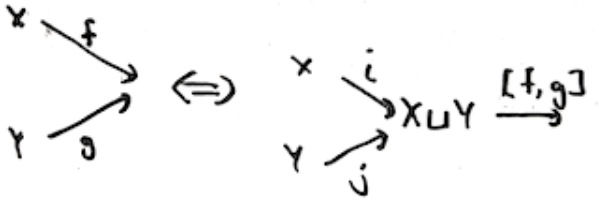
[f, g]: X ⊔ Y → Z
[f, g] ∘ i = f

$[f, g] \circ j = g$



Universal property

Two arrows		Single arrow
$f: X \to Z$	<=>	$[f,g]: X \sqcup Y \to Z$
$g: Y \to Z$		$f = [f,g] \circ i \quad g = [f,g] \circ j$



Proposition: bifunctor
$\sqcup$ is a **bifunctor**
i.e. if A is a category with coproducts

$\sqcup: A \times A \to A$
$s \sqcup t: X \sqcup Y \to X' \sqcup Y'$
$s \sqcup t = [iX' \circ s, jY' \circ t]$

Proposition: arrows
Let
$f: Z \to X \quad g: Z \to Y$
$s: X \to X' \quad t: Y \to Y'$

- Arrows

Any $h: X \sqcup Y \to Z$
$h: X \sqcup Y \to Z$
$h = [f, g]$

$[f, g] \circ i = h \circ i = f$
$[f, g] \circ j = h \circ j = g$

Special case: bifunctor

$s \sqcup t: X \sqcup Y \to X \sqcup Y'$

$s \sqcup t = [i' \circ s, j' \circ t]$

$$
\begin{aligned}
s \sqcup t \circ i &= [i' \circ s, j' \circ t] \circ i \\
&= i' \circ s \\
s \sqcup t \circ j &= [i' \circ s, j' \circ t] \circ j \\
&= j' \circ t
\end{aligned}
$$

- Compositions

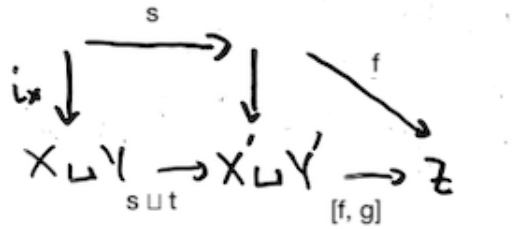
Basic

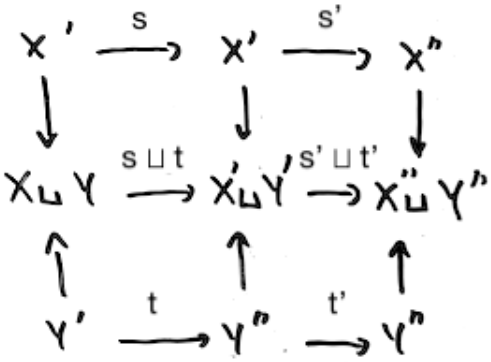
$$h \circ [f, g] = [h \circ f, h \circ g]$$

$$
\begin{aligned}
[f, g] \circ (s \sqcup t) &= [[f, g] \circ i \circ s, [f, g] \circ j \circ t] \\
&= [f \circ s, g \circ t]
\end{aligned}
$$

Bifunctor

$$
\begin{aligned}
(s' \sqcup t') \circ (s \sqcup t) &= [(s' \sqcup t') \circ i \circ s, (s' \sqcup t') \circ j \circ t] \\
&= [i' \circ s' \circ s, j' \circ t' \circ t] \\
&= (s' \circ s) \sqcup (t' \circ t)
\end{aligned}
$$





Finite products

Finite products

A category has finite products if it has products of every finite number, including 0 and 1

0-product (terminal)	**1-product (identity)**	**2-product**
....		
$0: X \to 0$	$1X: X \to X$	$p: XxX' \to X$
$0 =$ **terminal**		$q: XxX' \to X'$

Likewise, a finite coproduct

0-coproduct (initial)	**1-coproduct (identity)**	**2-coproduct**
....		
0: 0 → X	1X: X → X	i: X → X ⊔ X'
0 = **initial**		j: X' → X ⊔ X'

Theorem: finite products/coproducts
If a category has:

a) a **terminal element** and
b) **product XxY** for any object pair X, Y

then the category has all **finite products.**

Likewise, if

a) a **initial element** and
b) **coproduct X⊔Y** for any object pair X, Y

then the category has all **finite coproducts.**

Theorem:
If a category has finite products

associative
X x (Y x Z) ≅ (X x Y) x Z

reflexive
X x 0 ≅ X
0 x X ≅ X

==========
----- Equalizer/Coequalizer, Pushout/Pullback -----

Equalizer, coequalizer
Equalizer is a generalization of **kernel** for categories
Coequalizer is a generalization of **quotient** for categories.
i.e. a morphism e: Z → X such that

kernel **f ∘ e = 0 ∘ e** e: Ker f → X e.g. in Set with 0: X → {1}

Ker f = { x ∈ X: f(x) = 1 }

equalizer **f ∘ eq = g ∘ eq** eq: Eq(f,g) → X e.g. in Set

Eq (f, g) = { x ∈ X: f(x) = g(x) }

quotient **q ∘ f = q ∘ 0** q: Y → Y/fX e.g. in Set with 0: X → {1}

[y]

$\in Y/fX = [1]$ if $y \in fX$

$\{y\}$ if $y \notin fX$

coequalizer **coeq ∘ f = coeq ∘ g** coeq: Y → Coeq(f,g) e.g. in Set

Coeq (f, g) = Y / f ~ g

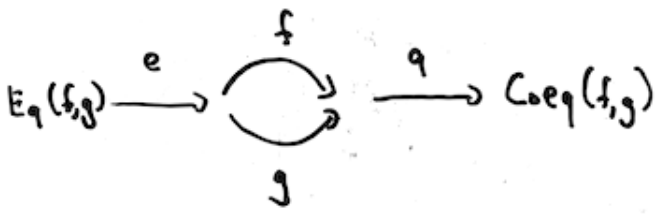
[y]

$\in Y/ f\sim g$ = [f(x)] = [g(x)] if $y \in f(X)$ or g(X)

= {y} otherwise

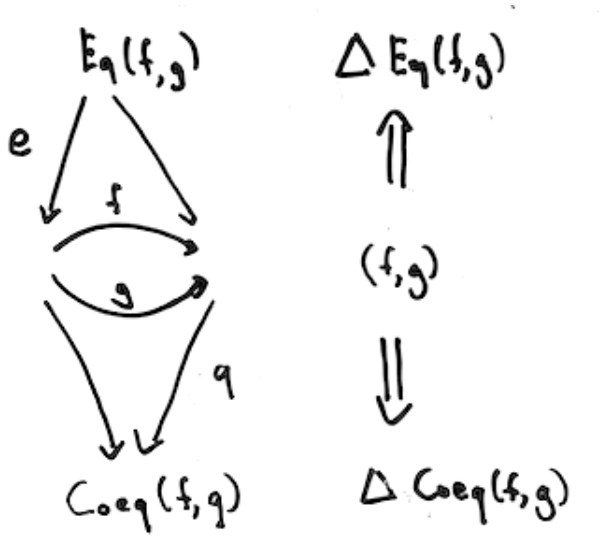
Definition
Let f, g: X → Y in category A
As **universal properties**:

	Equalizer Eq(f, g)	**Coequalizer Coeq(f, g)**
property	$f \circ \delta = g \circ \delta$	$\delta \circ f = \delta \circ g$
universal	f ∘ eq = g ∘ eq eq: Eq(f, g) → X	coeq ∘ f = coeq ∘ g coeq: Y → Coeq(f, g)



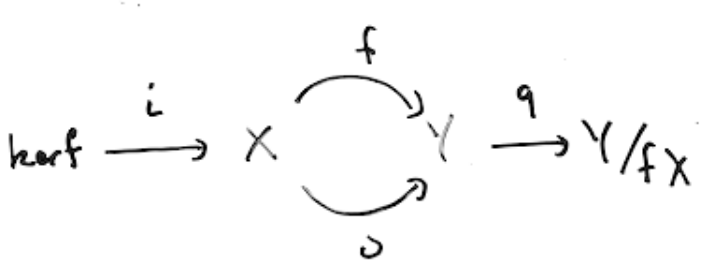
As **limits**:

	Eq(f, g)	**Coeq(f, g)**
shape	(f, g): ⇉ → A ΔZ ⇒ (f, g)	(f, g): ⇉ → A (f, g) ⇒ ΔZ
limit	Eq(f, g) = Lim (f, g) *eq: ΔEq(f, g) ⇒ (f, g)	Coeq(f, g) = Colim(f, g) q*: (f, g) ⇒ ΔCoeq(f, g)



Example: kernel and quotient in Set
Equalize f with the **0 map**
e.g. 0 = {1} in Grp

Eq (f, 0) = **ker f** **Coeq (f, 0)** = Y/fX
ker f = { x ∈ X: f(x) = 1 }



Limit: Pullback (X ×Z Y), Pushout (X ⊔Z Y)
Pullback X ×Z Y is a generalization of **product X x Y**
Both define **projections p: X x Y → X, q: X x Y → Y**
Product is unconditional, pullback is **conditional**

Product X x Y
projections
p: X x Y → X, q: X x Y → Y
no condition on X, Y

Pullback X xZ Y
projections
p: X xZ Y → X, q: X xZ Y → Y
condition f ∘ p = g ∘ q

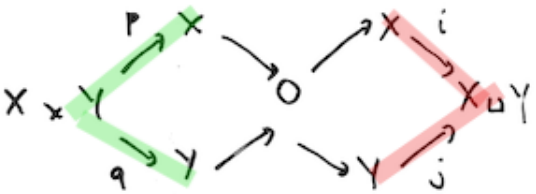
Pushout X ⊔Z Y is a generalization of **coproduct X ⊔ Y**

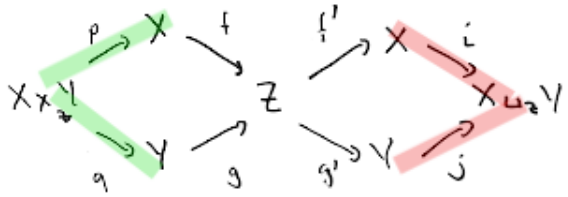
Coproduct
injections
i: X → X ⊔ Y, j: Y → X ⊔ Y

Pushout X ⊔Z Y
injections
i: X → X ⊔Z Y, j: Y → X ⊔Z Y

no condition on X, Y

condition i ∘ f = j ∘ g





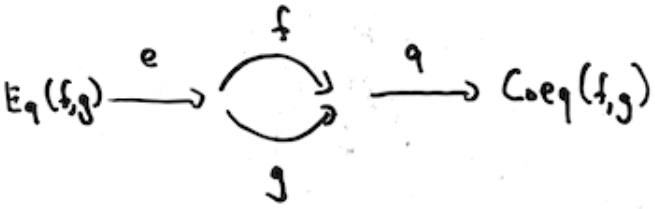
Generalization of equalizer/coequalizer to **X x Y** and **X ⊔ Y** (in Set)

Equalizer
f, g: X → Y
f ∘ e = g ∘ e
Eq(f, g) = { x ∈ X | f(x) = g(x) }
⊆ X

Pullback
f: X → Z, g: Y → Z
f ∘ i = g ∘ j
X xZ Y = { (x, y) ∈ UxV | f(x) = g(y) }
⊆ XxY

Coequalizer
f, g: X → Y
q ∘ f = q ∘ g
Coeq(f,g) = Y/ f ~ g

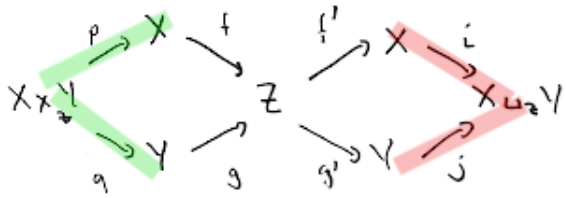
Pushout
f': Z → X, g': Z → Y
p ∘ f' = q ∘ g'
X ⊔Z Y = X ⊔ Y / f' ~ g'



Definition
Let f: X → Z, g: Y → Z
As universal properties:

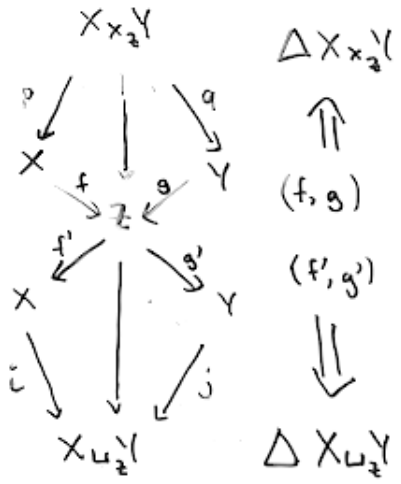
	Pullback X ×Z Y	**Pushout X ⊔Z Y**
property	f: X → Z, g: Y → Z f ∘ δ = g ∘ δ'	f': Z → X, g': Z → Y δ ∘ f' = δ' ∘ g'
universal	f ∘ i = g ∘ j	p ∘ f' = q ∘ g'
	injections	**projections**

i: X xZ Y → X p: X → X ⊔Z Y
j: X xZ Y → Y q: Y → X ⊔Z Y



As **limits**:

	Pullback	**Pushout**
shape	(f, g): (→←) → A *(δ, δ'): **Δ**W ⇒ (f, g)	(f', g'): (←→) → A (δ, δ')*: (f, g) ⇒ **Δ**W
limit	X ×Z Y = Lim (f, g) *(p, q): **Δ**(X xZ Y) ⇒ (f, g)	X ⊔Z Y = Colim (f', g') (i, j)*: (f', g') ⇒ **Δ**(X ⊔Z Y)



Example: Set
Sets always have pullbacks

X xZ Y = { (x, y) ∈ X x Y: f(x) = g(y) }
⊆ X x Y

X ⊓Z Y = X ⊓ Y / f ~g
t ∈ X ⊓ Y / f ~g = {x} if x ∈ X \ fZ
{y} if y ∈ Y \ gZ
[f(z)] = [g(z)]

==========
Proof

- **pullback**:
Suppose f ∘ u = g ∘ v: W → Z

=> define
h: W → X xZ Y

h(w) = (u(w), v'(w)) exists since f(u(w)) = g(v(w))

=> ph (w) = u(w) qh(w) = v(w)
h unique by definition

- **pushout**:
Suppose u ∘ f = v ∘ g: Z → W

=> define h by just picking a **representative**
h: X ⊔Z Y → W
h([t]) = u(t) if there exist t ∈ X
v(t) if there exist t ∈ Y

If h is well-defined, then by definition, h is **unique** and

h(i(z)) = u(z) g(j(z)) = v(z)

claim: f~g preserves values of u and v
i.e.
f(z) ~ f(z') => u(f(z)) = u(v(z'))
g(z) ~ g(z') => v(g(z)) = v(g(z'))

****
Proof claim:
Two cases by induction on **transitivity**:

a) f(z) ~ g(z) = g(z') ~ f(z') => u(f(z)) = v(g(z)) = v(g(z')) = u(f(z'))

b) g(z) ~ f(z) = f(z') ~ g(z) => u(f(z)) = u(f(z'))

=> preserved by transitivity

****

claim: h is well-defined i.e. independent of choice of representative
There are 3 possible types of t

****
Proof claim:
t = { x ∈ X } => h(t) = u(x) well-defined

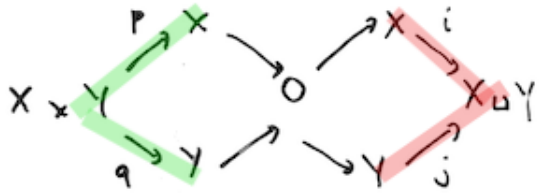
t = { y ∈ Y } => h(t) = v(y) well-defined

t = [f(z)] for some z => well-defined by preservation claim above
****

QED
==========

Example: X x Y, X ⊔ Y
Pullback from a **terminal element** 0 is the product X x Y
Pushout from an **initial element 0** is the coproduct X ⊔ Y

Pullback	Pushout
X x0 Y = X x Y	**X ⊔0 Y = X ⊔ Y**
projection	injection
p: X x Y → X	i: X → X ⊔ Y
q: X x Y → Y	j: Y → X ⊔ Y



==========
Proof
- **X x Y**
Suppose 0 is a terminal element
Let f: Z → X, g: Z → Y. Then

0x ∘ f: Z → 0 0y ∘ g: Z → 0

=> there exists unique α: Z → X x0 Y
f = pα g = qα

=> X x0 Y is the same property as X x Y so X x Y ≅ X x0 Y

QED
==========

Example: kernel/cokernel pair
Pullback and pushout from the **same function**

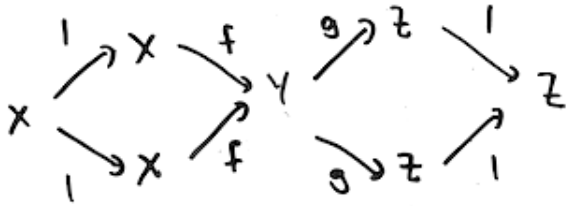
Kernel pair (p, q) of f	**Cokernel pair (i, j)** of g
X xY X = { (x, x') ∈ X x X: f(x) = f(x') } in Set	Z ⊔ Y Z = Z ⊔ Z / f ~ f in Set

Special case: **diagonal** pairs
i.e. **identity pairs**

Diagonal kernel pair (1X, 1X)	Diagonal cokernel pair
X xY X ≅ X	**Z ⊔Y Z ≅ Z**
***(1X, 1X)**: ΔX ⇒ (f, f)	**(1Z, 1Z)***: (g, g) ⇒ ΔZ

Then

f: X → Y is **monic**	<=>	pullback of (f, f) is **diagonal**
g: Y → Z is **epic**	<=>	pushout of (g, g) is **diagonal**

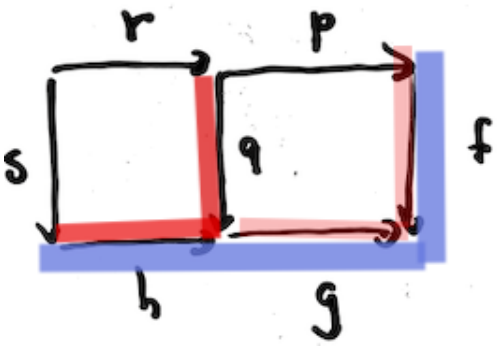


Proposition (composition)
1) **outer**: If

f ∘ p = g ∘ q pullback of (f, g) with **projections (p, q)**
q ∘ r = h ∘ s pullback of (q, h) with **projections (r, s)**

then

f ∘ (p ∘ r) = (g ∘ h) ∘ s pullback of (f, g ∘ h) with **projections (p ∘ r, s)**

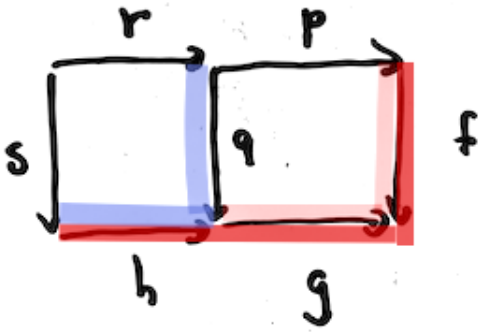


2) **left**: If

f ∘ p = g ∘ q pullback of (f, g) with **projections (p, q)**
f ∘ (p ∘ r) = (g ∘ h) ∘ s pullback of (f, g ∘ h) with **projections (p ∘ r, s)**

then

q ∘ r = h ∘ s pullback of (q, h) with **projections (r, s)**



Proposition (via equalizer)
Generalization of Sets:

X ×Z Y = { (x, y) ∈ X x Y: f(x) = g(y) } i.e. defined by product XxY + equalize f(x) = g(y)
X ⊓Z Y = X ⊓ Y / f ~ g i.e. defined by coproduct X ⊓Z Y + coequalizer f(z) ~ g(z)

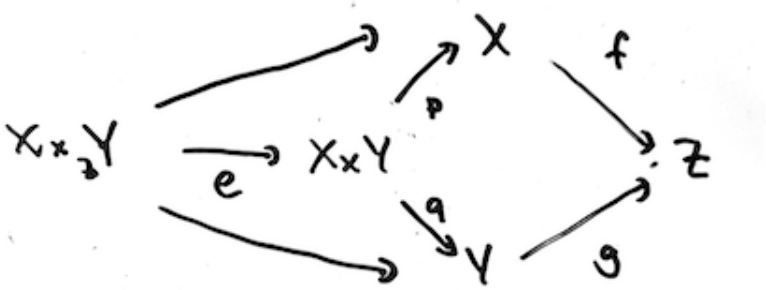
- **pullbacks**: if a category has **products** and **equalizers**, then it has pullbacks
i.e. Pullback of (f: X → Z, g: Y → Z) is

f ∘ (p ∘ e) = g ∘ (q ∘ e) with projections
p ∘ e: X ×Z Y → X q ∘ e: X ×Z Y → Y

where
p: X x Y → X q: X x Y → X

projections of X x Y

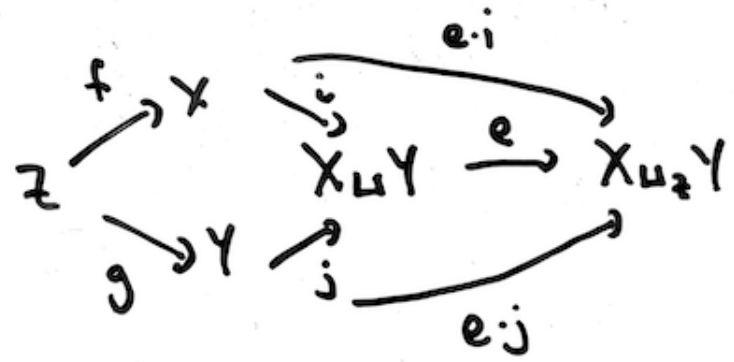
e: X xZ Y → XxY equalizer of (f ∘ p, g ∘ q): XxY → Z



- **pushouts**: if a category has **coproducts** and **coequalizers**, then it has pushouts
i.e. Pushouts of (f: Z → X, g: Z → Y) is

(e ∘ i) ∘ f = (e ∘ j) ∘ g with injections
e ∘ i: X → X ⊔Z Y e ∘ j: Y → X ⊔Z Y
where
i: X → X ⊔ Y j: Y → X ⊔ Y injections of X ⊔ Y

e: X ⊔ Y → X ⊔Z Y coequalizer of (i ∘ f, j ∘ g): Z → X ⊔ Y



==========

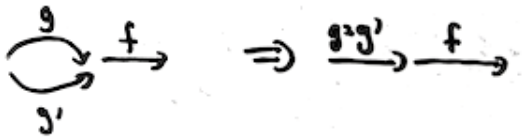
---- Limit Examples ----

Limit Examples

M**onic** f: X → Y
injection for morphisms

For any g, g'

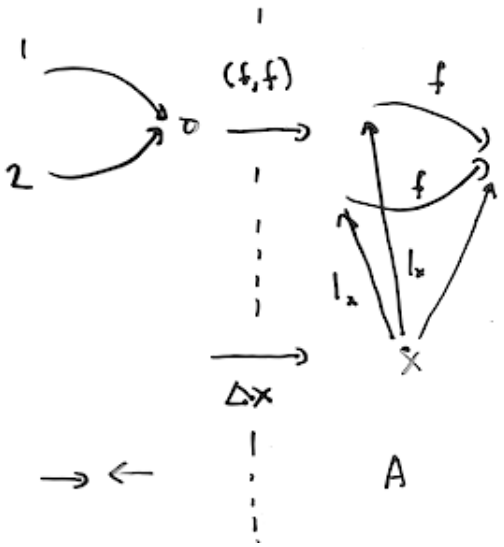
fg = fg' <=> g = g'



Limit definition: pullback →← with trivial universal cone (1X, 1X)

Shape
(f, f): (→←) → A
(f, f)→ = f (f, f)←= f

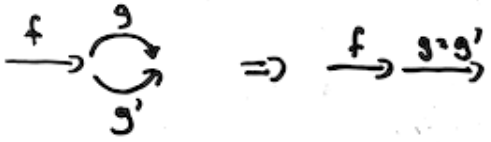
Universal cone
(1X, 1X): **Δ**Lim (f, f) ⇒ (f, f)
= **Δ**X



Epic f: X → Y
Surjection for morphisms

Definition:
For any g, g'

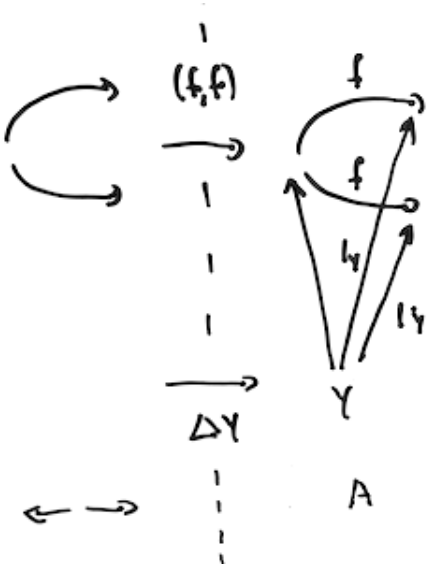
gf = g'f <=> g = g'



Limit definition: pushout ←→ with trivial universal cone (1Y, 1Y)

Shape
(f, f): (←→) → A
(f, f)← = f (f, f)→ = f

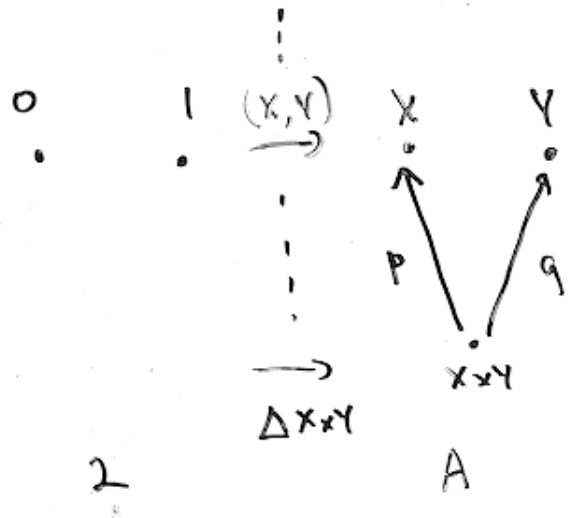
Universal cone
(1Y, 1Y): (f, f) ⇒ **Δ**Lim (f, f)
= **Δ**Y



Product X ⊓ Y (aka cartesian product X x Y, direct product X⊗Y)
Direct product X x Y for categories

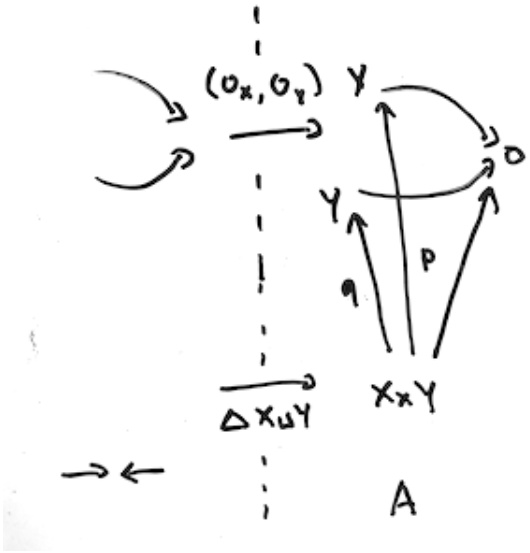
Limit definition: universal cone of **2** (· ·)

Shape	Universal cone
(X, Y): (· ·)→ A	(p, q) : ΔLim (X, Y) ⇒ (X, Y)
	= ΔX x Y



Equivalently, pullback →← to a **terminal object 0**

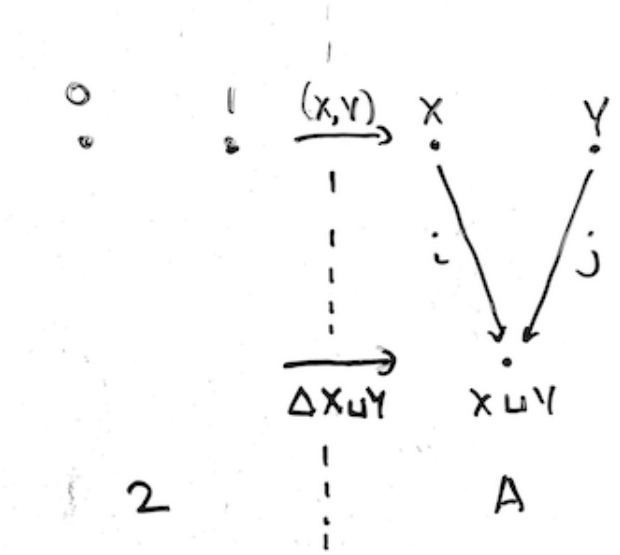
Shape	Cone
(0X, 0Y): (→←) → A	(p, q): ΔLim (0X, 0Y) → (0X, 0Y)
0 = terminal	= ΔX x Y



Coproduct X ⊔ Y (aka disjoint union X⊎Y, direct sum X⊕Y)
Disjoint union X⊎Y for categories

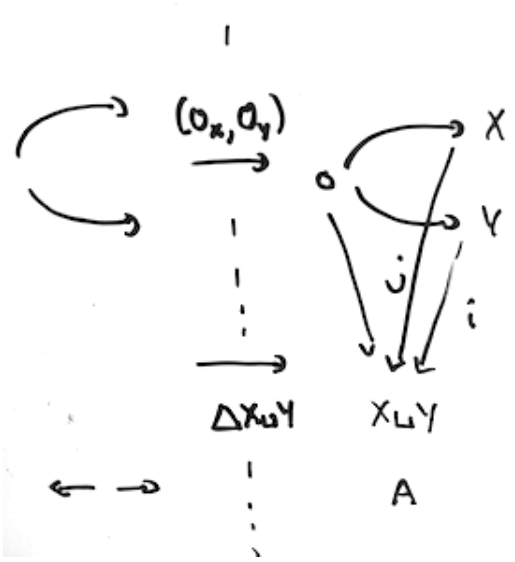
Limit definition: couniversal cone of **2** (· ·)

Shape	Couniversal cone
(X, Y): (0 · 1) → A	(i, j): (X, Y) ⇒ **Δ**Colim (X, Y)
	= **Δ**X ⊔ Y



Equivalently, a pushout ←→ from an **initial object 0**

Shape	**Couniversal cone**
(0X, 0Y): (←→) → A	(i, j): (X, Y) → **Δ**Colim (X, Y)
0 = initial	= **Δ**X⊔Y



Kernel pair of f: X → Y
Generalization of **injection**.
Also a dual **coequalizer**

f ∘ u = f ∘ v	kernel pair (u, v) of f
e ∘ f = e ∘ g	coequalizer e of (f, g)

Special cases

ker pair f = (1X, 1X)	**injection**

Limit: pullback →←

Shape	Limit
(f, f): (→←) → A	[u, v]: ΔLim (f, f) ⇒ (f, f)
	fu = fv

Cokernel pair of f: X → Y
Generalization of **surjection**.
Also a dual **equalizer**

u ∘ **f = v** ∘ **f**	cokernel pair (u, v) of f
f ∘ e = g ∘ e	equalizer e of (f, g)

Special cases

ker pair f = (1X, 1X)	**surjection**

Limit: pullback ←→

Shape	Limit
(f, f): (←→) → A	[u, v]: ΔLim (f, f) ⇒ (f, f)
	uf = vf

==========
======

----- OLD -----

OLD

Proposition: compositions
Let

η: F ⇒ F' where F, F': A → B

ε: G ⇒ G' where G, G': B → C

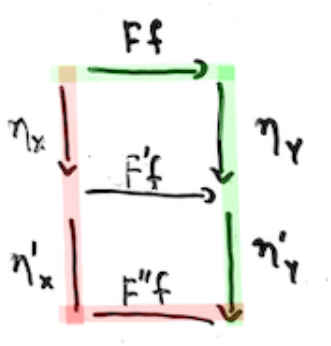
1) **vertical composition**: F ⇒ F' ⇒ F''

Composition in the **same category B**
Let η': F' ⇒ F''

$\eta' \cdot \eta$ **: F ⇒ F''**

$\eta'\eta$X: FX → F''X

$\eta'\eta$X = η'X ηX



2) **horizontal composition**: GF ⇒ G'F'/GF' ⇒ G'F'

Composition **across multiple categories B → C**

$\varepsilon \circ \eta$**: GF ⇒ G'F'**

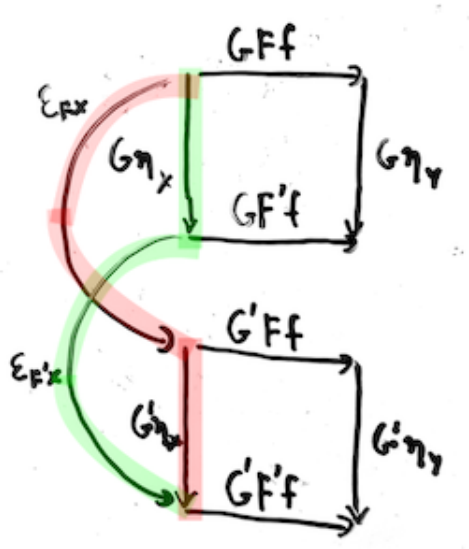
$\varepsilon \circ \eta$X: GFX → G'F'X

$\varepsilon \circ \eta$ = ε**F'** $\cdot$ **G**η as GF ⇒ GF' ⇒ G'F'

= **G'**$\eta \cdot \varepsilon$**F** as GF ⇒ G'F ⇒ G'F'

= ($\varepsilon \circ$ 1**F')** $\cdot$ **(1G** $\circ \eta$) as whiskering

= (1**G'** $\circ \eta$) $\cdot$ **(**$\varepsilon \circ$ **1F)**



3) **whiskering**: special case of horizontal composition with 1
Let K: I → A, L: C → J
Then

ηK: FK ⇒ F'K
ηKX = ηFKX

Lη: LF ⇒ LF
LηX = LηX

As horizontal composition

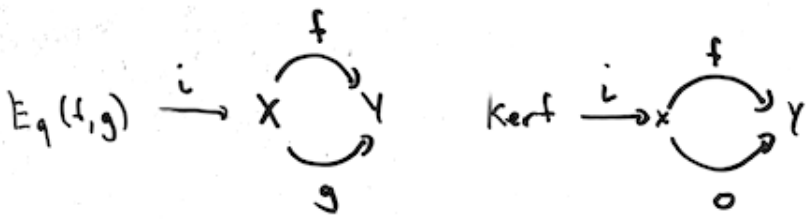
ηK = η ∘ 1K
Lη = 1F ∘ η

4)

Equalizers, pullbacks

Equalizer

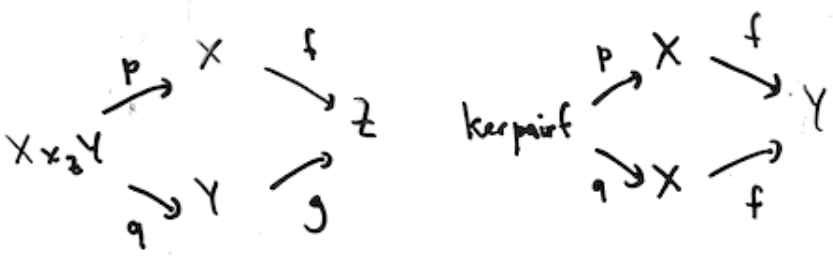
Eq (f, g)	=	Lim (f, g) (f, g): ⇉ → A
	=	{ x ∈ X: f(x) = g(x) }
Ker f	=	Eq (f, 0)
	=	{ x ∈ X : f(x) = 0 }



Pullback

X xZ Y	=	Lim (f, g) (f, g): (→←) → A
	=	{ (x, y) ∈ XxY : f(x) = g(y) }

Ker pair f = X xZ X
with (f, f)
= { (x, x') ∈ XxX: f(x) = f(x') }



Vertical composition
Composition of transformations **within the same category B**.

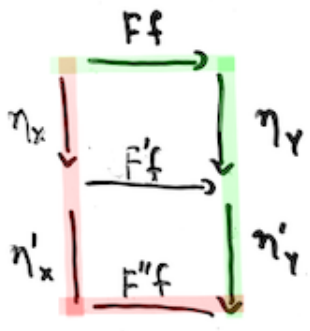
FA ⊆ B → F'A ⊆ B → F''A ⊆ B
η: F ⇒ F' η': F' ⇒ F''

Definition
Let η: F ⇒ F', η': F' ⇒ F''
Vertical composition transformation of η and ε is

$\eta'\eta$: F ⇒ F''
$\eta'\eta$X: FX → F''X component
$\eta'\eta$X = η'X ∘ ηX object X ∈ A

This satisfies naturality

$\eta'\eta$Y ∘ Fg = F''f ∘ $\eta'\eta$X f: X → Y



Horizontal composition
Composition of transformations **across different categories B => C**.

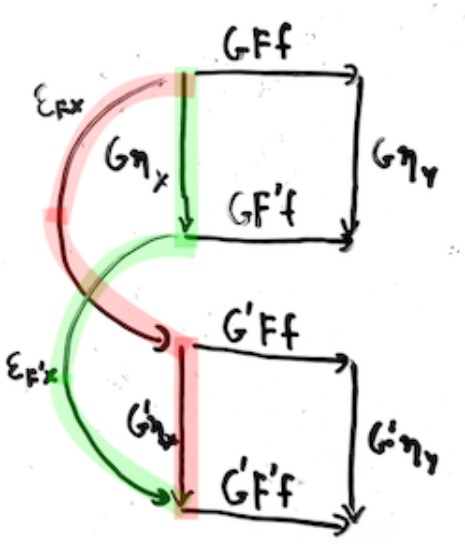
FA ⊆ B → F'A ⊆ B → GB ⊆ C
→ G'B ⊆ C
η: F ⇒ F' G: B → C ε: G
⇒ G'

Definition
Let
η: F ⇒ F' where F, F': A → B
ε: G ⇒ G' where G, G': B → C
The **horizontal composition transformation** η and ε is

$\varepsilon \circ \eta$: GF ⇒ G'F'
$(\varepsilon \circ \eta)$X: GFX → G'F'X component
$(\varepsilon \circ \eta)$X = εF'X ∘ GηX object X ∈ A
= G'ηX ∘ εFX

This satisfies naturality

$(\varepsilon \circ \eta)$Y ∘ GFf = G'F'f ∘ $(\varepsilon \circ \eta)$X f: X → Y



Whiskering
Whiskering: https://en.wikipedia.org/wiki/Natural_transformation#Whiskering
A type of **restriction** of a natural transformation by functors.
Let F, G: B → A
H: C → B
K: A → D
η: F → G

Define

Right whiskering by H i.e. **restriction** to H
ηH: FH ⇒ GH FH, GH: C → A

Left whiskering by K i.e. mapping by K
Kη: KF ⇒ KG KF, KG: B → D